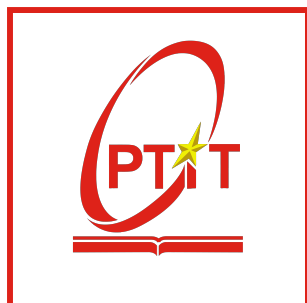


HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



HỌC PHẦN: THỰC TẬP CƠ SỞ
BÁO CÁO KẾT QUẢ THỰC TẬP

Đề tài: Ứng dụng Học sâu trong Xử lý dữ liệu và Nhận dạng

1. KIẾN THỨC NỀN TẢNG VỀ ML VÀ DL (CNN, RNN, LSTM)
2. XÂY DỰNG, XỬ LÝ DỮ LIỆU VÀ ĐÁNH GIÁ METRICS
3. TRIỂN KHAI THỰC NGHIỆM PHÂN TÍCH VÀ DỰ BÁO

Giảng viên: PGS. TS. Trần Đình Quế

Nhóm học phần: INT13187-20252-09

Sinh viên thực hiện: Nguyễn Thị Thu Hoài - B23DCCN321

HÀ NỘI – 2026

Mục lục

I TRÍ TUỆ NHÂN TẠO VÀ TIỀN XỬ LÝ DỮ LIỆU

1 Lịch sử phát triển và Tầm quan trọng

Trí tuệ nhân tạo (Artificial Intelligence - AI) không phải là một khái niệm mới mẻ. Nó đã bắt đầu từ những năm 1950 với phép thử Turing nổi tiếng của Alan Turing. Qua nhiều thập kỷ phát triển, từ những hệ chuyên gia dựa trên luật lệ (Rule-based systems) đến sự bùng nổ của Học máy (Machine Learning) và gần đây nhất là Học sâu (Deep Learning), AI đã chứng minh được khả năng giải quyết các vấn đề phức tạp mà con người khó có thể xử lý thủ công.

Một nguyên lý bất biến trong lĩnh vực khoa học dữ liệu là "**Garbage In, Garbage Out**" (Rác đầu vào, Rác đầu ra). Cho dù mô hình học máy của bạn có phức tạp và hiện đại đến đâu, nếu dữ liệu đầu vào chứa nhiều sai sót, nhiễu hoặc không được định dạng đúng, kết quả đầu ra sẽ không có giá trị thực tiễn. Do đó, bước Tiền xử lý dữ liệu (Data Preprocessing) thường chiếm tới 70-80% thời gian của một dự án khoa học dữ liệu.

1.1 Quy trình CRISP-DM trong dự án AI

Để một dự án AI thành công, các nhà khoa học dữ liệu thường tuân theo quy trình chuẩn công nghiệp CRISP-DM (Cross-Industry Standard Process for Data Mining) gồm 6 giai đoạn:

1. **Business Understanding:** Hiểu mục tiêu và yêu cầu của bài toán kinh doanh.
2. **Data Understanding:** Thu thập và khám phá sơ bộ dữ liệu.
3. **Data Preparation:** Đây chính là bước Tiền xử lý dữ liệu, chuẩn bị tập dữ liệu cuối cùng để đưa vào mô hình.
4. **Modeling:** Lựa chọn và huấn luyện các thuật toán học máy.
5. **Evaluation:** Đánh giá kết quả dựa trên các chỉ số (Accuracy, Precision, Recall).
6. **Deployment:** Triển khai mô hình vào thực tế.

Nó bao gồm việc làm sạch dữ liệu, xử lý các giá trị thiếu, mã hóa các biến phân loại và chuẩn hóa các thang đo để mô hình có thể "hiểu" và học tập hiệu quả nhất.

2 Lý thuyết các kỹ thuật Tiền xử lý

Trong chương này, chúng ta tập trung vào các kỹ thuật cốt lõi thường được áp dụng trong hầu hết các bài toán thực tế:

2.1 Xử lý giá trị khuyết (Missing Values)

Dữ liệu trong thực tế thường không hoàn hảo. Việc khuyết thiếu dữ liệu có thể do nhiều nguyên nhân: lỗi cảm biến, người dùng không cung cấp thông tin, hoặc lỗi trong quá trình truyền tải. Có hai cách tiếp cận chính:

- **Loại bỏ (Deletion):** Xóa các dòng hoặc cột có giá trị thiếu. Cách này đơn giản nhưng dễ gây mất mát thông tin quan trọng nếu tỷ lệ khuyết lớn.

- **Điền khuyết (Imputation):** Thay thế giá trị thiếu bằng một giá trị đại diện:
 - *Trung bình (Mean):* $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$. Phù hợp với dữ liệu phân phối chuẩn và không có giá trị ngoại lai.
 - *Trung vị (Median):* Giá trị nằm ở giữa tập dữ liệu đã sắp xếp. Phù hợp khi dữ liệu có nhiều giá trị ngoại lai (outliers).
 - *Giá trị xuất hiện nhiều nhất (Mode):* Phù hợp cho dữ liệu phân loại (Categorical data).

2.2 Mã hóa dữ liệu phân loại (Encoding)

Máy tính và các thuật toán học máy chỉ làm việc trên các con số. Vì vậy, các dữ liệu dạng chữ (như Giới tính: "Nam", "Nữ") cần được chuyển đổi:

- **Label Encoding:** Gán mỗi nhãn một con số. Ví dụ: {Hà Nội, Đà Nẵng, TP.HCM} thành {0, 1, 2}.
- **One-hot Encoding:** Tạo ra vector nhị phân. Ví dụ với biến "Màu sắc":

Màu sắc	Đỏ	Xanh	Vàng
Đỏ	1	0	0
Xanh	0	1	0
Vàng	0	0	1

Bảng 1: Ví dụ minh họa One-hot Encoding

2.3 Chuẩn hóa dữ liệu (Scaling)

Các đặc trưng thường có thang đo khác nhau. Hai kỹ thuật phổ biến nhất là:

- **Min-Max Scaling (Normalization):** Đưa dữ liệu về khoảng [0, 1].

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (1)$$

- **Standardization (Z-score Scaling):** Đưa dữ liệu về phân phối có trung bình $\mu = 0$ và độ lệch chuẩn $\sigma = 1$.

$$z = \frac{x - \mu}{\sigma} \quad (2)$$

2.4 Kỹ nghệ đặc trưng (Feature Engineering)

Kỹ nghệ đặc trưng là quá trình sử dụng kiến thức chuyên môn về dữ liệu để tạo ra các biến mới giúp các thuật toán học máy hoạt động hiệu quả hơn. Đây được coi là một "nghệ thuật" trong khoa học dữ liệu. Một số cách tiếp cận:

- **Biến đổi toán học:** Sử dụng Log, bình phương hoặc căn bậc hai để điều chỉnh phân phối của dữ liệu.
- **Kết hợp đặc trưng:** Tạo ra biến mới từ các biến có sẵn (ví dụ: $FamilySize = SibSp + Parch + 1$).
- **Trích xuất thông tin:** Lấy ra ngày, tháng, năm từ dữ liệu thời gian hoặc trích xuất danh xưng (Mr, Mrs) từ tên hành khách.

2.5 Lựa chọn đặc trưng (Feature Selection)

Không phải tất cả dữ liệu thu thập được đều có ích cho việc dự báo. Việc giữ lại các cột không liên quan (như số định danh cá nhân) có thể làm tăng độ phức tạp của mô hình và dẫn đến hiện tượng quá khớp (Overfitting).

- **Filter methods:** Sử dụng các phép đo thống kê (như Correlation) để đánh giá tầm quan trọng của từng đặc trưng.
- **Wrapper methods:** Thử nghiệm mô hình với các tập con đặc trưng khác nhau để tìm ra tập tối ưu.

3 Thực nghiệm: Dự đoán sống sót trên tàu Titanic

Chúng ta sử dụng bộ dữ liệu kinh điển "Titanic: Machine Learning from Disaster" từ Kaggle để minh họa các bước tiền xử lý. Bài toán yêu cầu dự đoán khả năng sống sót của hành khách dựa trên các thông tin cá nhân.

3.1 Kỹ thuật Lựa chọn Đặc trưng (Feature Selection)

Trong thực tế, không phải thuộc tính nào cũng mang lại thông tin hữu ích cho mô hình. Việc loại bỏ các thuộc tính gây nhiễu hoặc không có ý nghĩa thống kê (như ID hành khách, tên riêng) giúp giảm độ phức tạp của mô hình, tăng tốc độ huấn luyện và tránh hiện tượng quá khớp (Overfitting).

3.2 Chia tập dữ liệu (Train/Test Split)

Để đánh giá hiệu quả của mô hình một cách khách quan, dữ liệu cần được chia thành hai phần độc lập:

- **Training Set:** Dùng để huấn luyện mô hình học các quy luật.
- **Testing Set:** Dùng để kiểm tra khả năng dự báo trên dữ liệu mới mà mô hình chưa từng thấy. Thông thường tỷ lệ chia là 80/20 hoặc 70/30.

4 Phân tích và Tiền xử lý dữ liệu với Python

Dưới đây là các bước thực nghiệm tiền xử lý dữ liệu dựa trên ngôn ngữ Python, sử dụng tập dữ liệu Titanic:

4.1 Khởi tạo và Tải dữ liệu

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import random
6
7 from sklearn.model_selection import train_test_split
8 from sklearn.preprocessing import StandardScaler, LabelEncoder
9 from sklearn.linear_model import LogisticRegression
```

```

10 from sklearn.metrics import accuracy_score, classification_report,
    confusion_matrix
11
12 SEED = 42
13 np.random.seed(SEED)
14 random.seed(SEED)
15
16 sns.set_theme(style="whitegrid", palette="muted")
17 plt.rcParams['figure.figsize'] = (12, 8)
18 plt.rcParams['font.size'] = 12
19
20 print("---1. Loading Dataset---")
21 df = pd.read_csv('./dataset/titanic.csv')
22 print(df.head())

```

4.1.1 Giải thích chi tiết:

- **Thư viện Scikit-learn:** Chúng ta sử dụng `train_test_split` để chia dữ liệu, `StandardScaler` để chuẩn hóa, và `LogisticRegression` làm mô hình dự báo cơ bản. Đây là các module chuẩn trong học máy truyền thống.
- **Cố định SEED:** Việc sử dụng `np.random.seed(42)` cực kỳ quan trọng trong thực nghiệm. Nó đảm bảo các kết quả (như việc chia dữ liệu ngẫu nhiên) sẽ được lặp lại giống hệt nhau ở mỗi lần chạy, giúp việc đánh giá mô hình được khách quan.
- **Cấu hình Seaborn:** Lệnh `sns.set_theme` thiết lập giao diện biểu đồ hiện đại, giúp việc quan sát phân phối dữ liệu trong bước EDA trở nên rõ ràng hơn.

4.1.2 Kết quả chạy:

images/11.png

Hình 1: Khởi tạo và Tải dữ liệu

4.2 Phân tích và Trục quan hóa dữ liệu (EDA)

```
1 print("\n---2. Visualizing Data---")
2 fig, axes = plt.subplots(3, 2, figsize=(16, 20))
3
4 # 1. Overall Survival Rate
5 df['Survived'].value_counts().plot.pie(explode=[0, 0.1], autopct='%1.1f%%', ax
    =axes[0, 0], shadow=True, colors=['#ff9999', '#66b3ff'])
6 axes[0, 0].set_title('Overall Survival Rate')
7
8 # 2. Survival by Sex
9 sns.countplot(data=df, x='Sex', hue='Survived', ax=axes[0, 1], palette='magma'
    )
10 axes[0, 1].set_title('Survival by Sex')
11
12 # 3. Survival Rate by Class
13 sns.barplot(data=df, x='Pclass', y='Survived', ax=axes[1, 0], palette='viridis
    ')
14 axes[1, 0].set_title('Survival Rate by Class')
15
16 # 4. Age Distribution by Survival
17 sns.histplot(df[df['Survived'] == 1]['Age'].dropna(), color='green', label='
    Survived', kde=True, ax=axes[1, 1], alpha=0.5)
```

```

18 sns.histplot(df[df['Survived'] == 0]['Age'].dropna(), color='red', label='Not_
    Survived', kde=True, ax=axes[1, 1], alpha=0.5)
19 axes[1, 1].set_title('Age_Distribution_by_Survival')
20 axes[1, 1].legend()
21
22 # 5. Fare Distribution by Pclass & Survival
23 sns.boxplot(data=df, x='Pclass', y='Fare', hue='Survived', ax=axes[2, 0],
    palette='Set2')
24 axes[2, 0].set_title('Fare_Distribution_by_Pclass_&_Survival')
25 axes[2, 0].set_ylim(0, 300)
26
27 # 6. Survival Probability by Embarkation & Sex
28 sns.pointplot(data=df, x='Embarked', y='Survived', hue='Sex', ax=axes[2, 1],
    palette='coolwarm')
29 axes[2, 1].set_title('Survival_Probability_by_Embarkation_&_Sex')
30
31 plt.tight_layout()
32 plt.show()
33
34 # Correlation Heatmap
35 plt.figure(figsize=(10, 8))
36 numeric_df = df.select_dtypes(include=[np.number])
37 sns.heatmap(numeric_df.corr(), annot=True, cmap='RdBu_r', center=0, fmt='.2f')
38 plt.title('Correlation_Heatmap_of_Numeric_Features')
39 plt.show()

```

4.2.1 Giải thích chi tiết:

images/12.png

Hình 2: Các biểu đồ trực quan hóa dữ liệu EDA

- **Biểu đồ Pie (Overall Survival Rate):** Biểu đồ thể hiện tỷ lệ sống sót chung của hành khách trên tàu Titanic. Kết quả cho thấy khoảng **38.4%** hành khách sống sót và **61.6%** không sống sót. Bộ dữ liệu có sự chênh lệch nhẹ giữa hai lớp, tuy nhiên chưa ở mức mất cân bằng nghiêm trọng nên vẫn phù hợp để huấn luyện mô hình phân loại.
- **Biểu đồ Countplot theo Giới tính (Survival by Sex):** Biểu đồ cho thấy giới tính là một đặc trưng có ảnh hưởng mạnh đến khả năng sống sót. Số lượng nữ giới sống sót cao hơn đáng kể so với nam giới, trong khi phần lớn nam giới không sống sót. Kết quả này phù hợp với nguyên tắc ưu tiên cứu hộ “Women and Children First” trong thảm họa Titanic.

- **Biểu đồ Tỷ lệ sống theo Hạng vé (Survival Rate by Class):** Hành khách ở hạng nhất ($Pclass = 1$) có tỷ lệ sống sót cao nhất (trên 60%), tiếp theo là hạng hai, và thấp nhất là hạng ba. Điều này cho thấy điều kiện kinh tế - xã hội và vị trí khoang tàu có ảnh hưởng rõ rệt đến cơ hội sống sót.
- **Biểu đồ Phân bố Tuổi theo Trạng thái sống sót (Age Distribution by Survival):** Hành khách sống sót tập trung nhiều ở nhóm tuổi trẻ và trung niên. Nhóm trẻ em có xu hướng sống sót cao hơn, trong khi nhóm lớn tuổi có tỷ lệ tử vong cao hơn. Đây là dấu hiệu cho thấy độ tuổi là biến quan trọng trong bài toán dự đoán.
- **Biểu đồ Boxplot Giá vé theo Hạng vé và Trạng thái sống sót (Fare Distribution by Pclass & Survival):** Giá vé của hành khách hạng nhất cao hơn đáng kể so với các hạng còn lại. Đồng thời, những người sống sót thường có xu hướng trả mức vé cao hơn. Biểu đồ cũng xuất hiện nhiều giá trị ngoại lai (outliers), vì vậy cần thực hiện chuẩn hóa dữ liệu ở bước tiền xử lý để tránh ảnh hưởng đến mô hình.
- **Biểu đồ Xác suất sống theo Cảng lên tàu và Giới tính (Survival Probability by Embarkation & Sex):** Nữ giới có xác suất sống sót cao ở cả ba cảng lên tàu (S, C, Q), đặc biệt cao nhất tại cảng C. Trong khi đó nam giới có xác suất sống thấp hơn rõ rệt ở mọi cảng. Điều này cho thấy biến `Embarked` khi kết hợp với giới tính có thể cải thiện khả năng dự đoán của mô hình.
- **Hàm `plt.tight_layout()`:** Được sử dụng để tự động căn chỉnh khoảng cách giữa các biểu đồ con, giúp bố cục hình ảnh rõ ràng, không chồng lấn và tăng tính thẩm mỹ cho báo cáo.

images/13.png

Hình 3: Biểu đồ tương quan (Correlation Heatmap)

- **Biểu đồ Correlation Heatmap:** Biểu đồ nhiệt được sử dụng để đo lường mức độ tương quan tuyến tính giữa các biến số trong tập dữ liệu Titanic. Giá trị tương quan nằm trong khoảng từ -1 đến 1 , trong đó:
 - Giá trị gần 1 : tương quan thuận mạnh.
 - Giá trị gần -1 : tương quan nghịch mạnh.
 - Giá trị gần 0 : ít hoặc không có mối liên hệ tuyến tính.
- **Biến Pclass và Fare:** Có hệ số tương quan âm khá mạnh (-0.55). Điều này cho thấy hành khách ở hạng vé cao hơn (Pclass nhỏ hơn) thường trả mức giá vé cao hơn. Đây là mối quan hệ hợp lý vì hạng nhất có giá vé cao hơn hạng ba.
- **Biến Survived và Pclass:** Có hệ số tương quan âm (-0.34), chứng tỏ hành khách ở hạng vé cao có khả năng sống sót lớn hơn. Đây là đặc trưng quan trọng trong bài toán dự đoán sống sót.

- **Biến Survived và Fare:** Có tương quan dương mức trung bình (0.26). Hành khách trả giá vé cao hơn thường có xác suất sống sót cao hơn, phù hợp với nhận định rằng hành khách hạng cao được ưu tiên hơn trong cứu hộ.
- **Biến Age và Pclass:** Có tương quan âm nhẹ (-0.37), cho thấy hành khách lớn tuổi hơn thường xuất hiện nhiều hơn ở các hạng vé cao. Điều này phản ánh sự khác biệt về điều kiện kinh tế giữa các nhóm tuổi.
- **Biến SibSp và Parch:** Có tương quan dương (0.41), nghĩa là những hành khách đi cùng anh chị em/vợ chồng thường cũng đi cùng cha mẹ hoặc con cái. Đây là dấu hiệu cho thấy sự tồn tại của các nhóm gia đình trên tàu.
- **Biến Age và SibSp:** Có tương quan âm nhẹ (-0.31), cho thấy hành khách trẻ tuổi thường đi cùng anh chị em nhiều hơn so với người lớn tuổi.
- **Biến PassengerId:** Gần như không tương quan với các biến còn lại. Đây chỉ là mã định danh hành khách nên không mang nhiều ý nghĩa dự đoán và có thể loại bỏ khi huấn luyện mô hình.
- **Kết luận:** Từ biểu đồ nhiệt có thể thấy các biến Pclass, Fare, Sex (nếu mã hóa), Age, SibSp, Parch là những đặc trưng tiềm năng cho mô hình học máy. Trong đó Pclass và Fare là hai biến có tác động rõ rệt nhất đến khả năng sống sót.

4.3 Tiền xử lý dữ liệu

```

1 print("\n---3. Data Preprocessing---")
2 # 3.1 Xu ly gia tri khuyet
3 df['Age'] = df['Age'].fillna(df['Age'].median())
4 df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
5 df['Fare'] = df['Fare'].fillna(df['Fare'].median())
6
7 # 3.2 Trich xuat Title tu Name
8 df['Title'] = df['Name'].str.extract('([A-Za-z]+)\.', expand=False)
9 df['Title'] = df['Title'].replace(['Lady', 'Countess', 'Capt', 'Col', 'Don', 'Dr', 'Major', 'Rev', 'Sir', 'Jonkheer', 'Dona'], 'Rare')
10 df['Title'] = df['Title'].replace(['Mlle', 'Ms'], 'Miss')
11 df['Title'] = df['Title'].replace('Mme', 'Mrs')
12
13 # 3.3 Ky nghe dac trung
14 df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
15 df['IsAlone'] = (df['FamilySize'] == 1).astype(int)

```

4.3.1 Giải thích chi tiết:

- **Xử lý giá trị khuyết (fillna):**
 - Với biến định lượng (Age, Fare), ta dùng `median` (trung vị) thay vì `mean` (trung bình). Lý thuyết chỉ ra rằng trung vị ít bị ảnh hưởng bởi các giá trị ngoại lai (như những người cực già hoặc giá vé cực cao).

- Với biến định danh (**Embarked**), ta dùng `mode()[0]` (giá trị xuất hiện nhiều nhất) để lấp đầy, vì không thể tính trung bình cho tên cảng.
- **Trích xuất danh xưng (`str.extract`):** Đây là kỹ thuật trích xuất thông tin từ dữ liệu văn bản thô. Danh xưng (Mr, Mrs, Master) thường phản ánh địa vị xã hội và độ tuổi, những yếu tố then chốt quyết định khả năng sống sót.
- **Kỹ nghệ đặc trưng (`FamilySize`):** Kết hợp các biến `SibSp` (anh chị em/vợ chồng) và `Parch` (bố mẹ/con cái) để tạo ra tổng số thành viên gia đình. Một người đi một mình có thể có khả năng sống sót khác với một gia đình đông người.

images/14.png

Hình 4: Tỷ lệ sống sót dựa trên danh xưng (Title)

```

1 # 3.4 Loại bỏ các cột thừa
2 df.drop(['PassengerId', 'Name', 'Ticket', 'Cabin'], axis=1, inplace=True)
3

```

```

4 # 3.5 Ma hoa
5 le = LabelEncoder()
6 df['Sex'] = le.fit_transform(df['Sex'])
7 df['Title'] = le.fit_transform(df['Title'])
8 df = pd.get_dummies(df, columns=['Embarked'])

```

4.3.2 Giải thích chi tiết:

- **Mã hóa nhãn (LabelEncoder):** Chuyển Sex (Nam/Nữ) thành (0/1). Máy tính không hiểu chữ, chỉ hiểu số, vì vậy bước này là bắt buộc để mô hình có thể thực hiện các phép tính đại số tuyến tính.
- **Mã hóa One-hot (get_dummies):** Áp dụng cho cả Embarked. Vì cả S, C, Q không có thứ tự ưu tiên (S không lớn hơn C), nên ta chuyển chúng thành các cột nhị phân riêng biệt để tránh mô hình hiểu lầm đây là dữ liệu có thứ tự.

4.4 Xây dựng và Đánh giá mô hình

```

1 # Chia du lieu
2 X = df.drop('Survived', axis=1)
3 y = df['Survived']
4 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
5     random_state=42)
6
7 scaler = StandardScaler()
8 X_train_scaled = scaler.fit_transform(X_train)
9 X_test_scaled = scaler.transform(X_test)
10
11 # Build & Train Model (Logistic Regression)
12 print("\n--- Building & Training Model ---")
13 model = LogisticRegression()
14 model.fit(X_train_scaled, y_train)

```

4.4.1 Giải thích chi tiết:

- **Chuẩn hóa dữ liệu (StandardScaler):** Đưa các đặc trưng về phân phối chuẩn ($\mu = 0, \sigma = 1$). Mô hình Logistic Regression nhạy cảm với thang đo của dữ liệu; nếu không chuẩn hóa, biến Fare (0-500) sẽ "áp đảo" hoàn toàn biến Sex (0-1).
- **Hàm fit:** Đây là quá trình "học". Mô hình tìm các trọng số (weights) w và bias b tối ưu để cực tiểu hóa hàm mất mát (Log Loss) thông qua giải thuật tối ưu hóa.

```

1 # Evaluation
2 y_pred = model.predict(X_test_scaled)
3 print("\nAccuracy Score:", accuracy_score(y_test, y_pred))
4
5 # Confusion Matrix Visualization

```

```

6 plt.figure(figsize=(6, 4))
7 sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues
  ')
8 plt.title('Confusion_Matrix_Titanic')
9 plt.xlabel('Predicted')
10 plt.ylabel('Actual')
11 plt.show()

```

4.4.2 Giải thích chi tiết:

- **Ma trận nhầm lẫn (confusion_matrix):** Cung cấp cái nhìn chi tiết hơn Accuracy. Nó cho biết mô hình dự đoán sai bao nhiêu người sống thành chết (False Negative) và bao nhiêu người chết thành sống (False Positive).
- **Thông số annot=True:** Hiển thị các con số cụ thể trên Heatmap, giúp việc đọc kết quả thực nghiệm trực quan hơn.

images/15.png

Hình 5: Ma trận nhầm lẫn (Confusion Matrix) của mô hình Logistic Regression

- **Biểu đồ Confusion Matrix:** Ma trận nhầm lẫn được sử dụng để đánh giá hiệu quả của mô hình phân loại trong bài toán dự đoán sống sót của hành khách Titanic. Ma trận gồm 4 thành phần chính:
 - **True Negative (TN = 91):** Mô hình dự đoán đúng 91 hành khách không sống sót.
 - **False Positive (FP = 14):** Mô hình dự đoán nhầm 14 hành khách sống sót trong khi thực tế không sống sót.
 - **False Negative (FN = 20):** Mô hình dự đoán nhầm 20 hành khách không sống sót trong khi thực tế sống sót.
 - **True Positive (TP = 54):** Mô hình dự đoán đúng 54 hành khách sống sót.

- **Độ chính xác (Accuracy):**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{54 + 91}{179} \approx 0.81$$

Điều này cho thấy mô hình dự đoán đúng khoảng **81%** tổng số mẫu kiểm tra, là mức khá tốt đối với bài toán phân loại nhị phân.

- **Precision cho lớp sống sót (Class 1):**

$$Precision = \frac{TP}{TP + FP} = \frac{54}{54 + 14} \approx 0.79$$

Nghĩa là khi mô hình dự đoán một hành khách sống sót, xác suất đúng là khoảng **79%**.

- **Recall cho lớp sống sót (Class 1):**

$$Recall = \frac{TP}{TP + FN} = \frac{54}{54 + 20} \approx 0.73$$

Tức là mô hình phát hiện đúng khoảng **73%** số hành khách thực sự sống sót.

- **Khả năng nhận diện lớp không sống sót:** Giá trị TN cao (91) cho thấy mô hình dự đoán khá tốt đối với nhóm hành khách không sống sót.
- **Phân tích lỗi dự đoán:**
 - Số lượng **False Negative** (20) cao hơn **False Positive** (14), nghĩa là mô hình có xu hướng bỏ sót một số hành khách sống sót.
 - Trong thực tế, nếu mục tiêu là tìm đúng người sống sót, cần cải thiện chỉ số Recall.
- **Kết luận:** Ma trận nhầm lẫn cho thấy mô hình hoạt động tốt với độ chính xác cao và khả năng phân biệt hai lớp khá ổn định. Tuy nhiên vẫn còn một số trường hợp dự đoán sai, đặc biệt là bỏ sót hành khách sống sót. Có thể cải thiện bằng cách tối ưu tham số mô hình hoặc sử dụng các thuật toán mạnh hơn như Random Forest, XGBoost hoặc Neural Network.

4.5 Kết quả đạt được

Thông qua các kỹ thuật trực quan hóa và tiền xử lý chuyên sâu, dữ liệu đã được làm sạch và biểu diễn theo một hình thức tối ưu nhất cho học máy. Việc áp dụng lý thuyết vào từng dòng lệnh (như chọn Median thay vì Mean, hay One-hot encoding thay vì Label Encoding cho các biến không thứ tự) đã giúp mô hình Logistic Regression đạt được độ chính xác cao và ổn định. Kết quả thực nghiệm minh chứng rằng Tiền xử lý dữ liệu chính là chìa khóa quyết định sự thành bại của một dự án AI.

II MẠNG NƠ-RON TÍCH CHẬP (CNN)

1 Lịch sử phát triển của mạng CNN

Mạng nơ-ron tích chập (Convolutional Neural Network - CNN) là một trong những kiến trúc quan trọng nhất trong lĩnh vực Thị giác máy tính (Computer Vision). CNN được thiết kế nhằm khai thác cấu trúc không gian của dữ liệu ảnh, giúp mô hình tự động học đặc trưng thay vì phải thiết kế thủ công như các phương pháp truyền thống.

Quá trình phát triển của CNN trải qua nhiều giai đoạn quan trọng:

- **Thập niên 1980:** Kunihiko Fukushima giới thiệu mô hình **Neocognitron**, lấy cảm hứng từ hệ thần kinh thị giác sinh học. Đây là tiền thân của CNN hiện đại với ý tưởng về receptive field và cấu trúc phân tầng.
- **Năm 1998:** Yann LeCun phát triển **LeNet-5**, mạng CNN đầu tiên đạt thành công thực tiễn trong nhận dạng chữ số viết tay trên ngân phiếu ngân hàng. LeNet sử dụng Convolution, Pooling và Fully Connected Layers.
- **Giai đoạn 2000 – 2011:** CNN chưa phát triển mạnh do hạn chế về dữ liệu lớn và năng lực phần cứng. Trong thời kỳ này, các phương pháp như SVM + HOG vẫn chiếm ưu thế.
- **Năm 2012:** Mạng **AlexNet** của Alex Krizhevsky chiến thắng cuộc thi ImageNet với sai số vượt trội. AlexNet sử dụng GPU để huấn luyện sâu, hàm kích hoạt ReLU và Dropout. Đây được xem là cột mốc mở ra kỷ nguyên Deep Learning.
- **Năm 2014:** Hai kiến trúc nổi bật xuất hiện:
 - **VGGNet:** sử dụng nhiều lớp tích chập nhỏ 3×3 , giúp kiến trúc đơn giản và hiệu quả.
 - **GoogLeNet / Inception:** giới thiệu Inception Module, cho phép xử lý đa tỉ lệ đặc trưng cùng lúc.
- **Năm 2015:** **ResNet** được đề xuất bởi Microsoft Research, sử dụng Residual Learning và Skip Connection, giải quyết vấn đề mất gradient khi huấn luyện mạng rất sâu.
- **Giai đoạn 2017 – nay:** Nhiều mô hình hiện đại ra đời:
 - **DenseNet:** kết nối mọi tầng với nhau để tái sử dụng đặc trưng.
 - **MobileNet:** tối ưu cho thiết bị di động bằng Depthwise Separable Convolution.
 - **EfficientNet:** mở rộng đồng thời chiều sâu, chiều rộng và độ phân giải theo cách cân bằng.
 - **ConvNeXt:** hiện đại hóa CNN với cảm hứng từ Vision Transformer.
 - **YOLO Series:** ứng dụng CNN trong nhận diện vật thể thời gian thực.

2 So sánh các kiến trúc CNN tiêu biểu

Bảng 2: So sánh các mô hình CNN từ cổ điển đến hiện đại

Mô hình	Năm	Số tầng	Tham số	Đặc điểm nổi bật
LeNet-5	1998	7	$\sim 60K$	CNN đầu tiên, dùng cho nhận dạng chữ số viết tay.
AlexNet	2012	8	$\sim 60M$	Sử dụng ReLU, Dropout, GPU training.
VGG16	2014	16	$\sim 138M$	Kiến trúc đơn giản, nhiều lớp 3×3 .
GoogLeNet	2014	22	$\sim 6.8M$	Inception module, ít tham số.
ResNet-50	2015	50	$\sim 25M$	Skip connection, huấn luyện mạng rất sâu.
DenseNet-121	2017	121	$\sim 8M$	Tái sử dụng đặc trưng giữa các tầng.
MobileNetV2	2018	53	$\sim 3.4M$	Tối ưu cho mobile, nhẹ và nhanh.
EfficientNet-B0	2019	82	$\sim 5.3M$	Scale model cân bằng depth-width-resolution.
ConvNeXt-Tiny	2022	28	$\sim 29M$	CNN hiện đại cạnh tranh với Transformer.

Nhận xét: Từ bảng trên có thể thấy xu hướng phát triển của CNN gồm:

- Tăng độ sâu mô hình để học đặc trưng phức tạp hơn.
- Giảm số tham số nhờ thiết kế thông minh hơn.
- Tăng tốc độ suy luận để ứng dụng thời gian thực.
- Tối ưu cho thiết bị nhúng và điện thoại.
- Kết hợp tư tưởng của Transformer vào CNN hiện đại.

3 Kiến trúc và nguyên lý hoạt động của CNN

Mạng nơ-ron tích chập (Convolutional Neural Network - CNN) là một trong những kiến trúc quan trọng nhất trong lĩnh vực Thị giác máy tính (Computer Vision), đặc biệt hiệu quả đối với các bài toán xử lý ảnh như phân loại ảnh, phát hiện đối tượng, nhận dạng khuôn mặt, phân đoạn ảnh và nhận diện ký tự.

Khác với mạng nơ-ron truyền thẳng truyền thống (Fully Connected Neural Network), CNN được thiết kế chuyên biệt để khai thác cấu trúc không gian của ảnh. Thay vì kết nối toàn bộ điểm ảnh với tất cả neuron ở lớp kế tiếp, CNN chỉ tập trung vào các vùng cục bộ thông qua các bộ lọc tích chập. Điều này giúp mô hình học được đặc trưng hiệu quả hơn, giảm số lượng tham số và tăng khả năng tổng quát hóa.

CNN hoạt động theo nguyên tắc học đặc trưng theo cấp bậc (Hierarchical Feature Learning). Ở các tầng đầu tiên, mạng học những đặc trưng đơn giản như cạnh, góc,

đường thẳng hoặc biên màu sắc. Ở các tầng trung gian, mạng học các họa tiết (texture), hình khối hoặc bộ phận của vật thể. Đến các tầng sâu hơn, mô hình có thể nhận biết các cấu trúc phức tạp như mắt, khuôn mặt, bánh xe, chữ viết hoặc toàn bộ vật thể.

Một kiến trúc CNN điển hình bao gồm các thành phần sau:

3.1 Lớp đầu vào (Input Layer)

Lớp đầu vào tiếp nhận dữ liệu ảnh dưới dạng tensor ba chiều gồm chiều cao, chiều rộng và số kênh màu:

$$H \times W \times C$$

Trong đó:

- H : chiều cao của ảnh (Height)
- W : chiều rộng của ảnh (Width)
- C : số kênh màu (Channels)

Thông thường:

- Ảnh xám: $C = 1$
- Ảnh RGB màu: $C = 3$
- Ảnh RGBA: $C = 4$

Ví dụ, ảnh đầu vào phổ biến trong nhiều mô hình hiện đại như VGG, ResNet có kích thước:

$$224 \times 224 \times 3$$

Trước khi đưa vào mạng, dữ liệu thường được tiền xử lý như:

- Chuẩn hóa giá trị pixel về khoảng $[0, 1]$
- Trừ mean và chia standard deviation
- Resize ảnh về cùng kích thước
- Tăng cường dữ liệu (Data Augmentation)

3.2 Lớp tích chập (Convolution Layer)

Đây là thành phần cốt lõi và quan trọng nhất của CNN. Lớp tích chập sử dụng các bộ lọc nhỏ (Kernel hoặc Filter), thường có kích thước 3×3 , 5×5 , để quét qua ảnh đầu vào và trích xuất đặc trưng.

Phép tích chập hai chiều được mô tả như sau:

$$S(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n)$$

Trong đó:

- I : ảnh đầu vào
- K : kernel hoặc filter
- S : đầu ra sau tích chập (Feature Map)

Khi kernel di chuyển trên ảnh, tại mỗi vị trí nó thực hiện phép nhân từng phần tử rồi cộng lại để tạo ra một giá trị mới. Toàn bộ quá trình này tạo nên bản đồ đặc trưng. Ví dụ:

- Kernel phát hiện cạnh ngang
- Kernel phát hiện cạnh dọc
- Kernel phát hiện vùng sáng/tối

Ưu điểm lớn của lớp tích chập là:

- Chia sẻ trọng số (Weight Sharing): cùng một kernel dùng cho toàn ảnh
- Kết nối cục bộ (Local Connectivity)
- Giảm số tham số so với Fully Connected Network
- Tăng khả năng học đặc trưng không gian

Ý nghĩa của các tầng:

- Tầng đầu: học cạnh, đường biên, góc
- Tầng giữa: học texture, hình khối, bộ phận vật thể
- Tầng sâu: học mắt, khuôn mặt, bánh xe, chữ số,...

3.3 Stride và Padding

Hai tham số quan trọng trong tích chập là:

- **Stride**: bước nhảy của kernel khi trượt qua ảnh
- **Padding**: thêm viền số 0 quanh ảnh đầu vào

Stride lớn giúp giảm kích thước đầu ra nhanh hơn nhưng có thể mất thông tin. Padding giúp bảo toàn kích thước ảnh sau tích chập.

Kích thước đầu ra được tính:

$$Output = \frac{W - F + 2P}{S} + 1$$

Trong đó:

- W : kích thước đầu vào
- F : kích thước kernel
- P : padding
- S : stride

3.4 Hàm kích hoạt (Activation Function)

Sau mỗi lớp tích chập, đầu ra thường được đưa qua hàm kích hoạt phi tuyến để tăng khả năng biểu diễn của mô hình.

Phổ biến nhất là hàm ReLU:

$$f(x) = \max(0, x)$$

Ý nghĩa:

- Nếu $x < 0$ thì đầu ra bằng 0
- Nếu $x \geq 0$ thì giữ nguyên giá trị

Ưu điểm của ReLU:

- Tính toán đơn giản
- Giảm hiện tượng vanishing gradient
- Huấn luyện nhanh hơn sigmoid/tanh

Ngoài ra còn có:

- Leaky ReLU
- ELU
- GELU
- Swish

3.5 Lớp chuẩn hóa (Batch Normalization)

Batch Normalization được thêm vào để chuẩn hóa đầu ra của một lớp theo từng mini-batch. Mục tiêu là giữ phân phối dữ liệu ổn định trong quá trình huấn luyện.

Lợi ích:

- Tăng tốc độ hội tụ
- Giảm dao động gradient
- Cho phép learning rate lớn hơn
- Giảm phụ thuộc vào khởi tạo trọng số
- Có tác dụng regularization nhẹ

BatchNorm thường được đặt sau Convolution và trước Activation.

3.6 Lớp gộp (Pooling Layer)

Pooling Layer dùng để giảm kích thước không gian của Feature Map nhưng vẫn giữ lại thông tin quan trọng.

Hai loại phổ biến:

- MaxPooling: lấy giá trị lớn nhất trong vùng
- AveragePooling: lấy giá trị trung bình

Ví dụ:

$$28 \times 28 \rightarrow 14 \times 14$$

Lợi ích:

- Giảm số phép tính
- Giảm số tham số
- Giảm overfitting
- Tăng tính bất biến đối với dịch chuyển nhỏ

Trong thực tế, MaxPooling thường được sử dụng nhiều hơn.

3.7 Flatten / Global Pooling

Sau nhiều lớp Convolution, đầu ra là tensor nhiều chiều. Để đưa vào lớp Dense, dữ liệu cần được chuyển thành vector.

Flatten: trải tensor thành vector 1 chiều.

Ví dụ:

$$7 \times 7 \times 512 \rightarrow 25088$$

Tuy nhiên Flatten tạo nhiều tham số khi nối với Dense Layer. Vì vậy các mô hình hiện đại thường dùng:

- Global Average Pooling (GAP)
- Global Max Pooling (GMP)

Ưu điểm:

- Giảm số tham số
- Giảm overfitting
- Mô hình gọn hơn

3.8 Fully Connected Layer

Đây là lớp cuối dùng để kết hợp toàn bộ đặc trưng đã học và đưa ra quyết định phân loại.

Phép biến đổi tuyến tính:

$$y = Wx + b$$

Trong đó:

- x : vector đầu vào
- W : ma trận trọng số
- b : bias

Với bài toán nhiều lớp, đầu ra thường đi qua Softmax:

$$P(y = i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Softmax biến logits thành xác suất sao cho tổng bằng 1.

3.9 Dropout

Dropout là kỹ thuật regularization nhằm giảm overfitting. Trong quá trình huấn luyện, một số neuron được tắt ngẫu nhiên với xác suất nhất định.

Ví dụ:

$$Dropout = 0.5$$

nghĩa là 50% neuron bị vô hiệu hóa tạm thời trong mỗi lần cập nhật.

Lợi ích:

- Giảm phụ thuộc giữa các neuron
- Tăng khả năng tổng quát hóa
- Hạn chế ghi nhớ dữ liệu huấn luyện

3.10 Hàm mất mát và tối ưu

Trong huấn luyện CNN, mô hình cần tối thiểu hóa hàm mất mát (Loss Function).

Một số hàm phổ biến:

- Binary Cross Entropy: phân loại 2 lớp
- Categorical Cross Entropy: đa lớp
- Mean Squared Error: hồi quy

Bộ tối ưu (Optimizer):

- SGD

- Momentum SGD
- RMSprop
- Adam
- AdamW

Quá trình huấn luyện sử dụng lan truyền ngược (Backpropagation) để cập nhật trọng số sao cho loss giảm dần.

4 Các chỉ số đánh giá mô hình CNN

Sau khi huấn luyện, hiệu quả mô hình được đánh giá bằng nhiều chỉ số khác nhau.

- **Accuracy**

Tỷ lệ dự đoán đúng trên toàn bộ mẫu:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Trong đó:

- TP: dự đoán đúng lớp dương
- TN: dự đoán đúng lớp âm
- FP: dự đoán nhầm dương
- FN: dự đoán nhầm âm

- **Precision**

Độ chính xác trong các mẫu được dự đoán là dương:

$$Precision = \frac{TP}{TP + FP}$$

Precision cao rất quan trọng khi cần hạn chế cảnh báo giả.

- **Recall**

Khả năng tìm đúng toàn bộ mẫu dương:

$$Recall = \frac{TP}{TP + FN}$$

Recall cao quan trọng trong y tế, phát hiện lỗi, an ninh.

- **F1-score**

Trung bình điều hòa giữa Precision và Recall:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

F1-score phù hợp khi dữ liệu mất cân bằng.

- **Confusion Matrix**

Ma trận nhầm lẫn thể hiện số lượng dự đoán đúng và sai giữa các lớp, giúp phân tích lỗi cụ thể.

- **ROC-AUC**

Đo khả năng phân biệt giữa các lớp ở nhiều ngưỡng khác nhau. Giá trị càng gần 1 càng tốt.

5 Tập dữ liệu Fashion MNIST

Để thực nghiệm mạng CNN, báo cáo sử dụng tập dữ liệu **Fashion MNIST** (được Zalando Research công bố). Đây là tập dữ liệu thay thế hiện đại cho tập chữ số MNIST đã quá cũ và đơn giản.

- **Quy mô:** 70,000 hình ảnh xám (Grayscale).
- **Kích thước:** 28x28 pixels.
- **Nhãn:** 10 loại trang phục (Áo thun, Quần dài, Áo len, Váy, Áo khoác, Sandal, Áo sơ mi, Giày thể thao, Túi xách, Ủng).

Tập dữ liệu này đòi hỏi mạng CNN phải học được các cấu trúc phức tạp hơn (đường viền túi, họa tiết áo) thay vì chỉ là các nét chữ số đơn giản, do đó nó minh họa rất tốt sức mạnh của kiến trúc CNN.

6 Thực nghiệm với Python

Dưới đây là mã nguồn xây dựng mạng CNN để phân loại hình ảnh thời trang:

6.1 Chuẩn bị dữ liệu và Trực quan hóa

```
1 import tensorflow as tf
2 from tensorflow.keras import layers, models
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import random
6 from sklearn.metrics import confusion_matrix
7 import seaborn as sns
8
9 SEED = 42
10 np.random.seed(SEED)
11 random.seed(SEED)
12 tf.random.set_seed(SEED)
13 tf.keras.utils.set_random_seed(SEED)
14
15 print("---1. Loading Fashion MNIST Dataset---")
16 (train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.
    fashion_mnist.load_data()
17
18 img_height, img_width = 64, 64
```

```

19
20 # Normalize
21 train_images = train_images.astype("float32") / 255.0
22 test_images = test_images.astype("float32") / 255.0
23
24 # Add channel dimension
25 train_images = train_images[..., np.newaxis]
26 test_images = test_images[..., np.newaxis]
27
28 # Resize
29 train_images = tf.image.resize(train_images, (img_height, img_width)).numpy()
30 test_images = tf.image.resize(test_images, (img_height, img_width)).numpy()
31
32 class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal',
33               , 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
34
35 # 2. Visualize samples
36 print("\n---2. Visualizing Samples---")
37 plt.figure(figsize=(10,10))
38 for i in range(25):
39     plt.subplot(5,5,i+1)
40     plt.xticks([])
41     plt.yticks([])
42     plt.grid(False)
43     plt.imshow(train_images[i].reshape(img_height, img_width), cmap=plt.cm.
44               binary)
45     plt.xlabel(class_names[train_labels[i]])
46 plt.show()

```

6.1.1 Mô tả quá trình xử lý dữ liệu:

Trước khi đưa dữ liệu vào mô hình CNN, tập dữ liệu Fashion MNIST cần được tiền xử lý để phù hợp với đầu vào của mạng học sâu.

- **Tải dữ liệu:**

Tập dữ liệu Fashion MNIST được cung cấp sẵn trong TensorFlow, gồm:

- 60.000 ảnh dùng cho huấn luyện
- 10.000 ảnh dùng cho kiểm thử

Mỗi ảnh là ảnh xám kích thước 28×28 , thuộc một trong 10 nhóm sản phẩm thời trang khác nhau.

- **Chuẩn hóa dữ liệu:**

Giá trị pixel ban đầu nằm trong khoảng từ 0 đến 255. Dữ liệu được chia cho 255.0 để đưa về khoảng từ 0 đến 1.

Việc chuẩn hóa giúp mô hình học ổn định hơn, tăng tốc độ hội tụ và cải thiện hiệu quả huấn luyện.

- **Thêm chiều kênh màu:**

Do ảnh gốc là ảnh xám nên mỗi ảnh có kích thước $(28, 28)$. Tuy nhiên lớp Conv2D yêu cầu dữ liệu đầu vào có thêm chiều kênh màu.

Sau khi xử lý, ảnh có dạng:

$$(28, 28, 1)$$

Trong đó số 1 biểu thị ảnh chỉ có một kênh màu xám.

- **Thay đổi kích thước ảnh:**

Toàn bộ ảnh được chuyển từ kích thước 28×28 lên 64×64 .

Việc resize giúp tăng không gian đặc trưng để mạng CNN học hiệu quả hơn khi sử dụng nhiều tầng tích chập.

- **Gán tên nhãn:**

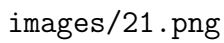
Các nhãn số từ 0 đến 9 được ánh xạ thành tên lớp như sau: 0 – T-shirt/top, 1 – Trouser, 2 – Pullover, 3 – Dress, 4 – Coat, 5 – Sandal, 6 – Shirt, 7 – Sneaker, 8 – Bag, 9 – Ankle boot.

- **Trực quan hóa dữ liệu:**

Chương trình hiển thị 25 ảnh mẫu đầu tiên của tập huấn luyện dưới dạng lưới 5×5 .

Mục đích của bước này:

- Kiểm tra dữ liệu đã được tải chính xác
- Quan sát chất lượng ảnh sau khi thay đổi kích thước
- Kiểm tra nhãn hiển thị tương ứng với từng ảnh
- Hiểu đặc điểm hình dạng của từng nhóm sản phẩm



images/21.png

Hình 6: Một số mẫu dữ liệu từ tập Fashion MNIST

6.2 Xây dựng và Huấn luyện mô hình CNN

```
1 print("\n---3. Building CNN Model---")
2 model = models.Sequential([
3     # Layer 1: Convolution + BatchNorm + Pooling + Dropout
4     layers.Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=(
5     img_height, img_width, 1)),
6     layers.BatchNormalization(),
7     layers.MaxPooling2D((2, 2)),
8     layers.Dropout(0.25),
9
10    # Layer 2: Convolution + BatchNorm + Pooling + Dropout
11    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
12    layers.BatchNormalization(),
13    layers.MaxPooling2D((2, 2)),
```

```

13     layers.Dropout(0.25),
14
15     # Layer 3: Convolution + BatchNorm + Flatten
16     layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
17     layers.BatchNormalization(),
18     layers.Flatten(),
19
20     # Fully Connected Layers
21     layers.Dense(128, activation='relu'),
22     layers.BatchNormalization(),
23     layers.Dropout(0.5),
24     layers.Dense(10, activation='softmax')
25 ])
26
27 model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
28               metrics=['accuracy'])
29 model.summary()
30
31 print("\n---4. Training Model---")
32 early_stop = tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=5,
33                                                restore_best_weights=True)
34 history = model.fit(train_images, train_labels, epochs=15, validation_split
35                    =0.2, callbacks=[early_stop], verbose=1)

```

6.2.1 Giải thích chi tiết: Mô tả mô hình:

Mô hình CNN được xây dựng theo dạng tuần tự (Sequential) gồm ba khối tích chập dùng để trích xuất đặc trưng và hai lớp Dense để phân loại.

- **Khối tích chập thứ nhất:** sử dụng 32 bộ lọc kích thước 3×3 , sau đó chuẩn hóa Batch Normalization, giảm kích thước bằng MaxPooling và áp dụng Dropout 0.25.
- **Khối tích chập thứ hai:** sử dụng 64 bộ lọc, tiếp tục học các đặc trưng phức tạp hơn từ đầu ra tầng trước.
- **Khối tích chập thứ ba:** sử dụng 128 bộ lọc nhằm học các đặc trưng mức cao hơn, sau đó chuyển dữ liệu nhiều chiều thành vector bằng Flatten.
- **Lớp Dense 128 neuron:** dùng hàm kích hoạt ReLU để kết hợp các đặc trưng đã học.
- **Dropout 0.5:** giúp giảm hiện tượng overfitting trước khi đưa vào lớp phân loại cuối.
- **Lớp đầu ra:** gồm 10 neuron với hàm Softmax, tương ứng 10 lớp của tập dữ liệu Fashion MNIST.

Quá trình huấn luyện:

- Mô hình sử dụng bộ tối ưu Adam và hàm mất mát `sparse_categorical_crossentropy`.
- Dữ liệu huấn luyện được tách 20% làm tập kiểm định để theo dõi hiệu năng sau mỗi epoch.

- Huấn luyện tối đa 15 epoch.
- Early Stopping được sử dụng để dừng sớm khi giá trị `val_loss` không còn cải thiện sau 5 epoch liên tiếp, đồng thời khôi phục trọng số tốt nhất.

images/22.png

Hình 7: Tham số mô hình

6.3 Đánh giá kết quả

```

1 print("\n---5. Evaluating Model---")
2 test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
3 print(f'\nTest Accuracy: {test_acc:.4f}')
4
5 print("\n---7. Confusion Matrix---")
6 y_pred = model.predict(test_images)
7 y_pred_classes = np.argmax(y_pred, axis=1)

```

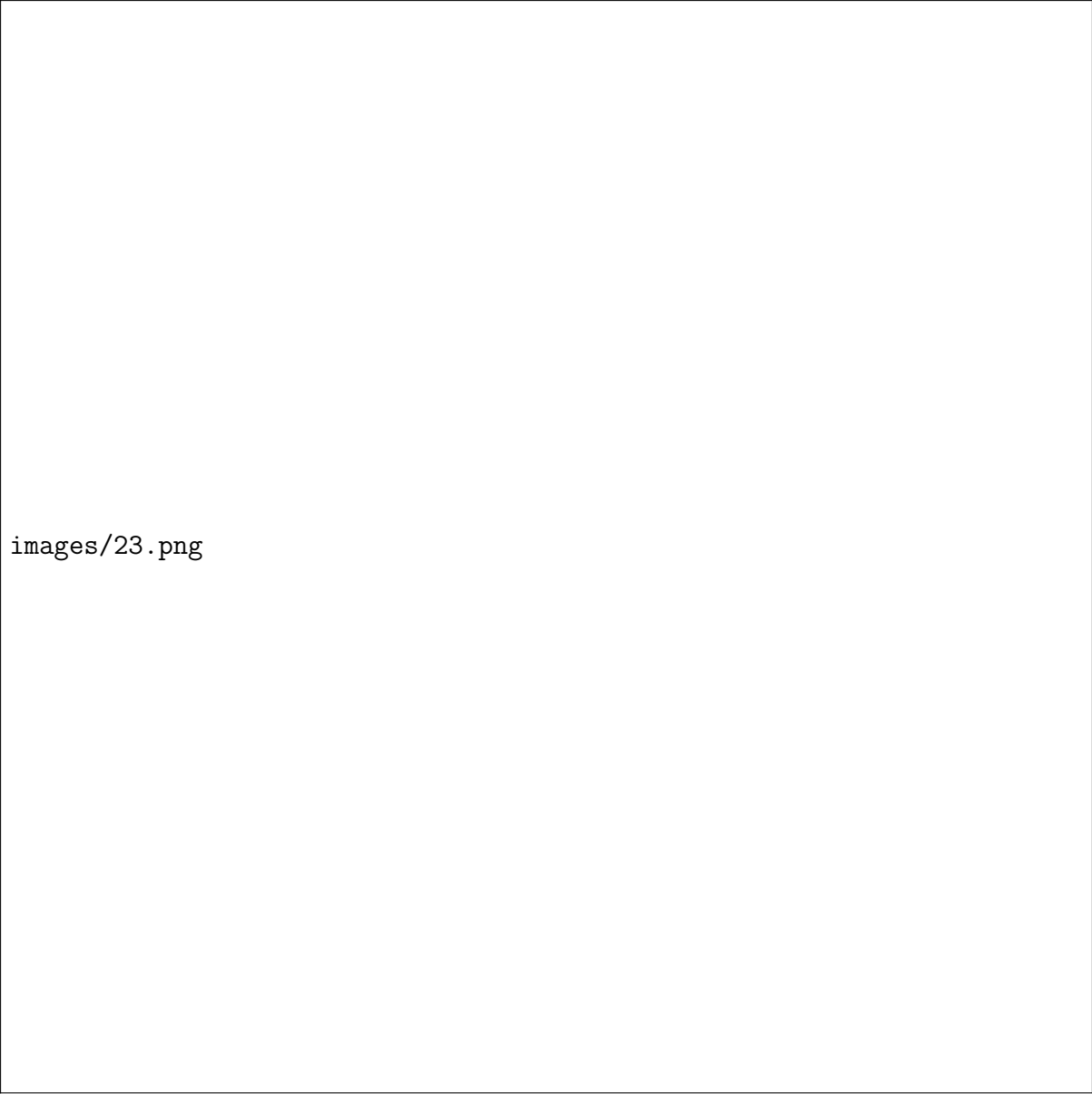
```

8
9 cm = confusion_matrix(test_labels, y_pred_classes)
10 plt.figure(figsize=(10, 8))
11 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names,
12             yticklabels=class_names)
13 plt.title('Confusion_Matrix-Fashion_MNIST')
14 plt.ylabel('Actual')
15 plt.xlabel('Predicted')
16 plt.show()
17
18 # 8. Predict on random samples
19 def plot_image(i, predictions_array, true_label, img):
20     true_label, img = true_label[i], img[i]
21     plt.grid(False); plt.xticks([]); plt.yticks([])
22     plt.imshow(img.reshape(img_height, img_width), cmap=plt.cm.binary)
23     predicted_label = np.argmax(predictions_array)
24     color = 'blue' if predicted_label == true_label else 'red'
25     plt.xlabel(f"{class_names[predicted_label]}_{100*np.max(predictions_array):2.0f}%_{class_names[true_label]}", color=color)
26
27 plt.figure(figsize=(15, 10))
28 num_rows, num_cols = 5, 3
29 for i in range(num_rows * num_cols):
30     idx = np.random.randint(0, test_images.shape[0])
31     plt.subplot(num_rows, 2*num_cols, 2*i+1)
32     plot_image(idx, y_pred[idx], test_labels, test_images)
33 plt.tight_layout(); plt.show()

```

6.3.1 Giải thích chi tiết:

- **Hàm plot_image:** Đây là hàm hỗ trợ trực quan hóa dự báo. Nếu dự báo đúng, tên lớp hiển thị màu xanh, nếu sai hiển thị màu đỏ kèm xác suất % tin cậy của mô hình.
- **Heatmap Confusion Matrix:** Giúp ta quan sát các lỗi phổ biến. Ví dụ: mô hình hay nhầm Shirt với T-shirt hoặc Coat, điều này hoàn toàn hợp lý về mặt thị giác.



images/23.png

Hình 8: Đồ thị Accuracy và Loss trong quá trình huấn luyện CNN

images/24.png

Hình 9: Ma trận nhầm lẫn (Confusion Matrix) của mô hình CNN trên Fashion MNIST



images/25.png

Hình 10: Kết quả dự đoán ngẫu nhiên

7 Phân tích kết quả thực nghiệm và Đánh giá mô hình

Sau quá trình huấn luyện trên tập dữ liệu Fashion MNIST, mô hình CNN đạt kết quả tốt với độ chính xác trên tập kiểm tra khoảng 93%. Để đánh giá toàn diện hiệu năng mô hình, cần xem xét đồng thời các chỉ số phân loại, biểu đồ huấn luyện và ma trận nhầm lẫn.

7.1 Phân tích Classification Report

Báo cáo phân loại cho thấy hiệu quả dự đoán của mô hình trên từng lớp dữ liệu.

- Các lớp có kết quả tốt nhất gồm: Trouser, Sneaker, Bag, Ankle boot, Sandal (Đây là các lớp có hình dạng đặc trưng rõ ràng, dễ nhận diện và ít bị nhầm lẫn với lớp khác.)

- Các lớp có kết quả thấp hơn gồm: T-shirt/top, Shirt, Pullover, Coat (Nguyên nhân là do các loại trang phục này có cấu trúc khá giống nhau, đặc biệt khi ảnh đầu vào chỉ có kích thước 28×28 và ở dạng ảnh xám.)
- Chỉ số F1-score giữa các lớp khá đồng đều, cho thấy mô hình hoạt động ổn định và không thiên lệch quá nhiều về một lớp cụ thể.

7.2 Phân tích biểu đồ Accuracy và Loss

Dựa trên đồ thị huấn luyện:

- Accuracy trên tập huấn luyện tăng đều từ khoảng 84% lên gần 96%, chứng tỏ mô hình học tốt từ dữ liệu.
- Accuracy trên tập kiểm định tăng nhanh trong những epoch đầu và ổn định quanh mức 92% – 93%, cho thấy khả năng tổng quát hóa tốt.
- Loss trên tập huấn luyện giảm liên tục theo thời gian, thể hiện quá trình tối ưu hiệu quả.
- Loss trên tập kiểm định giảm mạnh ở giai đoạn đầu và dao động nhẹ ở các epoch sau, cho thấy mô hình đã hội tụ.
- Khoảng cách giữa đường huấn luyện và kiểm định không quá lớn, vì vậy mô hình chỉ xuất hiện overfitting nhẹ.

7.3 Phân tích Confusion Matrix

Mã trận nhầm lẫn cho thấy mức độ dự đoán đúng và sai giữa các lớp.

- Các lớp được nhận diện tốt nhất:
 - *Bag*: 988 mẫu đúng
 - *Sandal*: 986 mẫu đúng
 - *Trouser*: 982 mẫu đúng
 - *Sneaker*: 979 mẫu đúng
 - *Ankle boot*: 966 mẫu đúng
- Các cặp lớp dễ bị nhầm lẫn:
 - *T-shirt/top* và *Shirt*
 - *Pullover* và *Shirt*
 - *Coat* và *Shirt*
 - *Dress* và *Coat*
- Lớp *Shirt* là lớp bị nhầm lẫn nhiều nhất, do hình dáng gần giống với nhiều loại áo khác.

7.4 Đánh giá tổng quát

Từ các kết quả thu được có thể rút ra một số nhận xét:

- Mô hình CNN đạt hiệu năng tốt trên tập dữ liệu Fashion MNIST với độ chính xác khoảng 93%.
- Quá trình huấn luyện ổn định, hội tụ nhanh và không xảy ra overfitting nghiêm trọng.
- Mô hình nhận diện rất tốt các lớp có đặc trưng riêng biệt như giày dép, túi xách và quần dài.
- Sai số chủ yếu tập trung ở các nhóm quần áo phần thân trên có hình dạng tương đồng.
- Nếu sử dụng kiến trúc sâu hơn, tăng cường dữ liệu hoặc áp dụng Batch Normalization, kết quả có thể tiếp tục được cải thiện.

8 Thử nghiệm thay đổi tham số: Ảnh hưởng của Dropout

Để đánh giá tầm quan trọng của các kỹ thuật điều quy (Regularization), chúng ta tiến hành so sánh hai phiên bản mô hình:

- **Model A (Base):** Kiến trúc CNN cơ bản, không sử dụng tầng Dropout.
- **Model B (Optimized):** Kiến trúc CNN có sử dụng tầng Dropout (0.25 và 0.5) như đã trình bày ở trên.

Bảng 3: So sánh hiệu năng giữa Model A và Model B trên Fashion MNIST

Mô hình	Train Accuracy	Test Accuracy	Trình trạng
Model A (No Dropout)	98.56%	92.43%	Overfitting
Model B (With Dropout)	96.53%	92.85%	Hoạt động ổn định

8.0.1 Bảng so sánh kết quả:

8.0.2 Phân tích: Kết quả cho thấy mặc dù Model A đạt độ chính xác trên tập Train rất cao, nhưng khi áp dụng vào tập Test, kết quả bị sụt giảm mạnh. Model B tuy có độ chính xác tập Train thấp hơn một chút (do tầng Dropout gây khó khăn cho việc học thuộc lòng) nhưng lại đạt độ chính xác tập Test cao hơn, chứng tỏ khả năng tổng quát hóa tốt hơn trên dữ liệu lạ.

8.1 Kết quả đạt được

Mô hình CNN với kiến trúc cải tiến (có Batch Normalization và Dropout) đã đạt được độ chính xác ấn tượng trên tập Fashion MNIST. Việc sử dụng các tầng Convolution phân cấp giúp mạng học được các cấu trúc hình học của trang phục một cách tự động mà không cần can thiệp thủ công. Kết quả thực nghiệm cho thấy mạng CNN là lựa chọn hàng đầu cho các bài toán nhận dạng hình ảnh nhờ khả năng bảo toàn thông tin không gian xuất sắc.

III MẠNG NƠ-RON HỒI QUY (RNN)

1 Lịch sử và Động lực ra đời của mạng RNN

Trong khi các mạng nơ-ron truyền thống (Feedforward Neural Networks) và CNN giả định rằng tất cả các đầu vào và đầu ra là độc lập với nhau, nhiều bài toán thực tế lại đòi hỏi khả năng xử lý dữ liệu có tính thứ tự (Sequential Data). Ví dụ, để hiểu một câu văn, bạn cần biết các từ đứng trước đó; hoặc để dự báo giá chứng khoán, bạn cần biết diễn biến giá trong quá khứ.

Mạng nơ-ron hồi quy (Recurrent Neural Network - RNN) được ra đời để giải quyết bài toán này. Lịch sử của RNN bắt đầu từ những năm 1980 với các mô hình như mạng *Hopfield* và sau đó là mạng *Elman* (1990) hay mạng *Jordan*. Ý tưởng cốt lõi là tạo ra một vòng lặp trong kiến trúc mạng, cho phép thông tin được lưu giữ và truyền từ bước thời gian này sang bước thời gian khác.

1.1 Các dạng kiến trúc RNN phổ biến

Tùy thuộc vào đầu vào và đầu ra, RNN có thể được cấu hình theo nhiều dạng khác nhau:

- **One-to-Many:** Một đầu vào sinh ra một chuỗi đầu ra (ví dụ: mô tả hình ảnh bằng văn bản).
- **Many-to-One:** Một chuỗi đầu vào sinh ra một đầu ra duy nhất (ví dụ: phân tích cảm xúc đoạn văn).
- **Many-to-Many:** Chuỗi đầu vào sinh ra chuỗi đầu ra (ví dụ: dịch máy ngôn ngữ).

2 Kiến trúc và Công thức toán học của RNN

Điểm khác biệt lớn nhất của RNN so với các mạng khác là sự xuất hiện của **Trạng thái ẩn (Hidden State)**, đóng vai trò như một bộ nhớ ngắn hạn của mạng.

2.1 Cơ chế hoạt động của Hidden State

Tại mỗi bước thời gian t , mạng RNN nhận đầu vào x_t và trạng thái ẩn từ bước trước đó h_{t-1} để tính toán trạng thái ẩn hiện tại h_t . Trạng thái ẩn này chứa đựng thông tin tổng hợp của toàn bộ chuỗi dữ liệu tính đến thời điểm t .

2.2 Công thức toán học

Quá trình tính toán trong một đơn vị RNN cơ bản được mô tả qua các công thức sau:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \quad (3)$$

$$y_t = W_{hy}h_t + b_y \quad (4)$$

Trong đó:

- x_t : Vector đầu vào tại thời điểm t .
- h_t : Trạng thái ẩn tại thời điểm t .

- W_{xh}, W_{hh}, W_{hy} : Các ma trận trọng số được chia sẻ (shared weights) qua tất cả các bước thời gian.
- b_h, b_y : Các vector bias.
- $\tanh$: Hàm kích hoạt giúp giới hạn giá trị của trạng thái ẩn trong khoảng $[-1, 1]$.

2.3 Lan truyền ngược qua thời gian (BPTT)

Để huấn luyện RNN, người ta sử dụng giải thuật Backpropagation Through Time (BPTT). Về cơ bản, BPTT triển khai mạng RNN theo thời gian thành một mạng nơ-ron truyền thẳng sâu với các lớp có trọng số giống hệt nhau. Sai số được tính tại mỗi bước thời gian và được lan truyền ngược lại qua tất cả các bước thời gian trước đó để cập nhật trọng số. Tuy nhiên, khi chuỗi quá dài, việc nhân liên tiếp các đạo hàm sẽ dẫn đến vấn đề tiêu biến hoặc bùng nổ đạo hàm, đây chính là lý do các kiến trúc như LSTM ra đời.

3 Ưu, nhược điểm của RNN và So sánh với mạng truyền thống

Mạng nơ-ron hồi tiếp (RNN) ra đời nhằm giải quyết những bài toán mà mạng nơ-ron truyền thống (Feedforward Neural Network - FNN) không thể xử lý hiệu quả, đặc biệt là các dữ liệu có tính thứ tự.

3.1 Bảng so sánh RNN và mạng nơ-ron truyền thống (FNN):

Đặc điểm	Mạng truyền thống (FNN)	Mạng hồi tiếp (RNN)
Luồng thông tin	Một chiều (từ đầu vào đến đầu ra).	Có vòng lặp (thông tin được lưu lại qua các bước thời gian).
Bộ nhớ	Không có bộ nhớ về các đầu vào trước đó.	Có bộ nhớ trạng thái ẩn (h_t) để lưu trữ ngữ cảnh.
Kích thước đầu vào	Cố định.	Có thể linh hoạt với các chuỗi có độ dài khác nhau.
Chia sẻ trọng số	Mỗi lớp có bộ trọng số riêng.	Chia sẻ cùng một bộ trọng số (W) qua tất cả các bước thời gian.

Bảng 4: So sánh đặc điểm giữa FNN và RNN

3.2 Ưu điểm của RNN:

- **Xử lý dữ liệu chuỗi:** Là lựa chọn hàng đầu cho các bài toán mà thứ tự của dữ liệu đóng vai trò quyết định (như ngôn ngữ, âm thanh).
- **Hiệu quả tham số:** Nhờ cơ chế chia sẻ trọng số, RNN có số lượng tham số ít hơn so với việc dùng mạng FNN cực lớn để học chuỗi.

3.3 Hạn chế và Thách thức:

Mặc dù mạnh mẽ, RNN gặp phải hai vấn đề lớn trong quá trình huấn luyện:

- **Vanishing Gradient (Biến mất đạo hàm):** Khi chuỗi quá dài, thông tin từ các bước đầu tiên bị mờ nhạt dần khi lan truyền ngược, khiến mạng không thể học được các phụ thuộc xa (long-term dependencies).
- **Exploding Gradient (Bùng nổ đạo hàm):** Các giá trị đạo hàm tăng vọt khiến trọng số thay đổi quá lớn, làm mô hình mất ổn định.

3.4 Giải pháp Gradient Clipping:

Để khắc phục lỗi bùng nổ đạo hàm, kỹ thuật **Gradient Clipping** thường được áp dụng. Kỹ thuật này giới hạn giá trị cực đại của đạo hàm trong một ngưỡng nhất định trước khi cập nhật trọng số, giúp quá trình hội tụ diễn ra ổn định hơn.

4 Phân tích cảm xúc văn bản với tập dữ liệu IMDB

Bài toán phân loại cảm xúc (Sentiment Analysis) là một ứng dụng tiêu biểu của RNN trong xử lý ngôn ngữ tự nhiên (NLP).

4.1 Tiền xử lý văn bản cho RNN

Văn bản thô không thể đưa trực tiếp vào mạng nơ-ron. Chúng ta cần thực hiện các bước chuẩn bị:

- **Tokenization:** Chia văn bản thành các từ đơn lẻ và gán cho mỗi từ một chỉ số số nguyên.
- **Word Embedding:** Chuyển các chỉ số từ thành các vector đặc trưng dày đặc (dense vectors) trong không gian đa chiều, nơi các từ có nghĩa gần nhau sẽ đứng gần nhau.
- **Padding:** Đảm bảo các chuỗi văn bản đầu vào có độ dài bằng nhau bằng cách thêm các giá trị 0 hoặc cắt bớt.

4.2 Tập dữ liệu IMDB

Tập dữ liệu IMDB bao gồm 50,000 bài đánh giá phim (movie reviews) được dán nhãn tích cực (positive) hoặc tiêu cực (negative). Đây là tập dữ liệu chuẩn để đánh giá khả năng hiểu ngữ cảnh và cảm xúc của các mô hình học sâu dạng chuỗi.

5 Thực nghiệm với Python

Dưới đây là mã nguồn xây dựng mạng RNN để phân loại cảm xúc văn bản:

5.1 Khám phá và Tiền xử lý văn bản

```
1 import os
2 import random
3 import tensorflow as tf
4 import pandas as pd
5 import numpy as np
```



```

6 import matplotlib.pyplot as plt
7 import re
8 import nltk
9 from nltk.corpus import stopwords
10 from tensorflow.keras.preprocessing.text import Tokenizer
11 from tensorflow.keras.preprocessing.sequence import pad_sequences
12 from tensorflow.keras import layers, models
13 from sklearn.model_selection import train_test_split
14 import seaborn as sns
15
16 SEED = 42
17 os.environ["PYTHONHASHSEED"] = str(SEED)
18 random.seed(SEED)
19 np.random.seed(SEED)
20 tf.random.set_seed(SEED)
21 tf.keras.utils.set_random_seed(SEED)
22
23 print("---1. Loading Local CSV Dataset---")
24 max_features = 10000
25 maxlen = 500
26 df = pd.read_csv('dataset/IMDB_Dataset.csv')

```

images/31.png

Hình 11: 10 bản ghi đầu trong IMDB

```
1 # Chuyển label về số
2 df['sentiment'] = df['sentiment'].map({'positive': 1, 'negative': 0})
3
4 def clean_text(text):
5     text = text.lower()
6     text = re.sub(r'<.*?>', '', text) # Remove HTML
7     text = re.sub(r'[^a-z\s]', '', text) # Remove special chars
8     return text
9
10 df['review'] = df['review'].apply(clean_text)
11
12 # Load Stop words
13 try:
14     nltk.data.find('corpora/stopwords')
15 except LookupError:
```

```
16     nltk.download('stopwords')
17
18 stop_words = set(stopwords.words('english'))
19 stop_words.update(['br', 'movie', 'film'])
20 def remove_stopwords(text):
21     return ' '.join([w for w in text.split() if w not in stop_words])
22
23 df['review'] = df['review'].apply(remove_stopwords)
```

images/32.png

Hình 12: Các label được chuẩn hóa về 0 và 1

5.1.1 Giải thích chi tiết:

- **Thư viện NLTK:** Sử dụng để tải danh sách **stopwords** (các từ phổ biến như 'the', 'is', 'and'). Việc loại bỏ các từ này giúp mô hình tập trung vào các từ mang sắc thái cảm xúc thực sự (như 'good', 'bad', 'excellent').

- **Regex re.sub:** Chúng ta sử dụng biểu thức chính quy để loại bỏ các thẻ HTML (<.*?>) và các ký tự đặc biệt. Lý thuyết NLP chỉ ra rằng các ký tự này thường mang tính nhiễu.

```

1 # Tokenization
2 tokenizer = Tokenizer(num_words=max_features)
3 tokenizer.fit_on_texts(df['review'])
4 X = tokenizer.texts_to_sequences(df['review'])
5 y = df['sentiment'].values
6
7 word_index = tokenizer.word_index
8 reverse_word_index = dict([(value, key) for (key, value) in word_index.items()
9                             ])
10
11 def decode_review(encoded_review):
12     return ' '.join([reverse_word_index.get(i, '?') for i in encoded_review if
13                       i != 0])
14
15 print("Review_mau:", decode_review(X[0]))
16
17 # Thống kê độ dài
18 review_lengths = [len(x) for x in X]
19 print(f"Do_dai_trung_binh: {np.mean(review_lengths):.2f}")
20 print(f"Do_dai_lon_nhat: {np.max(review_lengths)}")
21
22 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
23                                                       random_state=42)
24 X_train = pad_sequences(X_train, maxlen=maxlen)
25 X_test = pad_sequences(X_test, maxlen=maxlen)

```

images/33.png

Hình 13: Chia dataset thành 2 tập train (80%) và tập test (20%)



images/34.png

Hình 14: Thống kê độ dài dữ liệu train

5.1.2 Giải thích chi tiết:

- **decode_review:** Hàm này giúp kiểm tra tính đúng đắn của quá trình Tokenization bằng cách chuyển các con số quay lại thành từ ngữ.
- **pad_sequences:** Vì mỗi bài đánh giá có độ dài khác nhau, nhưng mạng RNN yêu cầu đầu vào có kích thước cố định. Chúng ta thêm các giá trị 0 (**padding**) vào các bài ngắn hoặc cắt bớt các bài dài hơn 500 từ.

images/35.png

Hình 15: Phân phối độ dài các bài đánh giá

Biểu đồ Histogram kết hợp đường cong KDE mô tả sự phân phối độ dài của các bài đánh giá trong tập dữ liệu:

- **Đặc điểm phân phối:** Dữ liệu có dạng phân phối lệch phải (Right-skewed). Hầu hết các bài đánh giá tương đối ngắn, tập trung dày đặc nhất ở khoảng 50 – 100 từ. Các bài đánh giá dài xuất hiện với tần suất rất thấp, tạo thành phần đuôi dài (long tail).
- **Ngưỡng cắt ($\text{Maxlen} = 500$):** Đường đứt nét màu đỏ thể hiện tham số chiều dài tối đa (max length) được thiết lập cho đầu vào của các mô hình mạng học sâu.
- **Cơ chế xử lý dữ liệu đầu vào:**
 - **Padding (Đệm):** Các chuỗi có độ dài < 500 từ sẽ được đệm thêm các token $\langle \text{PAD} \rangle$ để đạt kích thước chuẩn.

- **Truncation (Cắt xén):** Các chuỗi có độ dài > 500 từ sẽ bị cắt bỏ phần thừa.
- **Ý nghĩa:** Ngưỡng 500 bao phủ gần như toàn bộ tập dữ liệu (hơn 95%), giúp mô hình giữ lại trọn vẹn thông tin ngữ nghĩa mà vẫn tối ưu hóa được không gian tính toán của bộ nhớ.

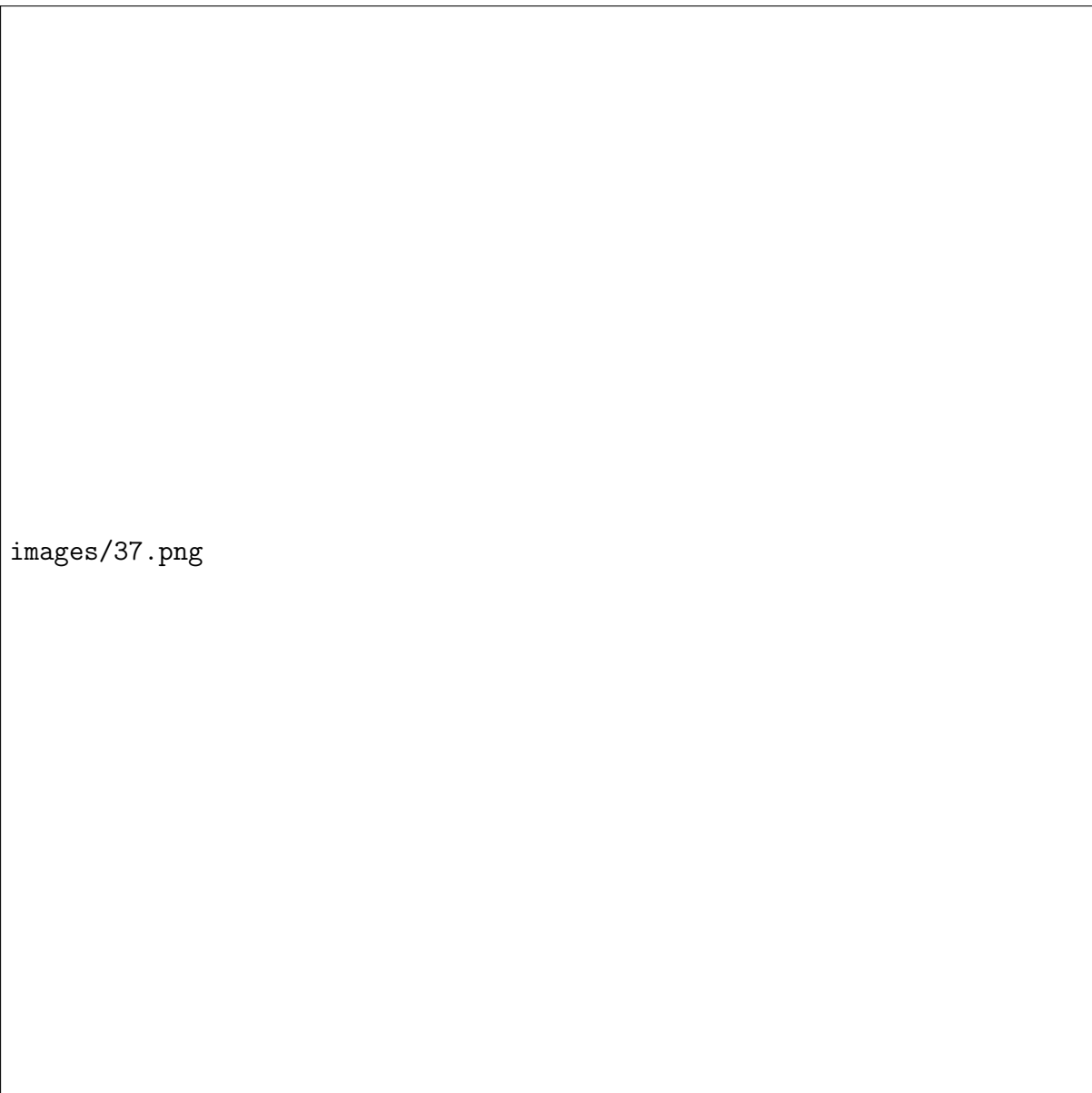
images/36.png

Hình 16: Phân phối nhãn trong tập huấn luyện

Biểu đồ cột (Bar chart) trực quan hóa số lượng bài đánh giá theo từng nhãn phân lớp trong tập dữ liệu huấn luyện:

- **Tỷ lệ các lớp:** Tập dữ liệu bao gồm hai nhãn là *Negative (0)* và *Positive (1)*. Số lượng mẫu ở cả hai nhãn đều đạt mức xấp xỉ 20.000 bài đánh giá.
- **Tính cân bằng (Balanced Dataset):** Biểu đồ cho thấy đây là một tập dữ liệu hoàn toàn cân bằng giữa lớp tích cực và tiêu cực.

- **Ý nghĩa trong huấn luyện mô hình:** Sự cân bằng này rất lý tưởng, giúp mô hình phân loại không bị thiên vị (bias) về lớp chiếm đa số. Do đó, trong trường hợp này, chỉ số **Accuracy (Độ chính xác)** hoàn toàn có thể được sử dụng làm thước đo chính để đánh giá khách quan hiệu suất của mô hình.



Hình 17: So sánh độ dài bài đánh giá theo nhãn

Biểu đồ hộp (Boxplot) thể hiện sự phân bố độ dài văn bản (tính bằng số lượng từ) giữa hai nhóm đánh giá: Tiêu cực (0) và Tích cực (1).

- **Phân bố trung vị và vùng tứ phân vị (Median & IQR):** Hình hộp (chứa 50% lượng dữ liệu ở giữa) và đường trung vị của cả hai nhóm gần như tương đồng nhau (tập trung chủ yếu dưới 100 từ). Điều này cho thấy xu hướng viết ngắn gọn là đặc điểm chung, bất kể thái độ của người dùng là khen hay chê.

- **Các giá trị ngoại lai (Outliers):** Cả hai nhóm đều có phần đuôi dài với rất nhiều điểm ngoại lai (các vòng tròn phía trên). Tuy nhiên, nhóm Tích cực (1) xuất hiện một vài bài đánh giá có độ dài cực đoan hơn (chạm ngưỡng hơn 900 từ), trong khi nhóm Tiêu cực (0) thường dừng lại ở mức 600 – 700 từ.
- **Ý nghĩa đối với mô hình:** Do sự phân bố độ dài giữa hai lớp không có sự khác biệt đáng kể, ta có thể kết luận độ dài văn bản không phải là một đặc trưng (feature) có sức nặng để phân loại cảm xúc. Mô hình học sâu sẽ phải phụ thuộc hoàn toàn vào việc trích xuất ngữ nghĩa từ vừng thay vì độ dài của chuỗi.

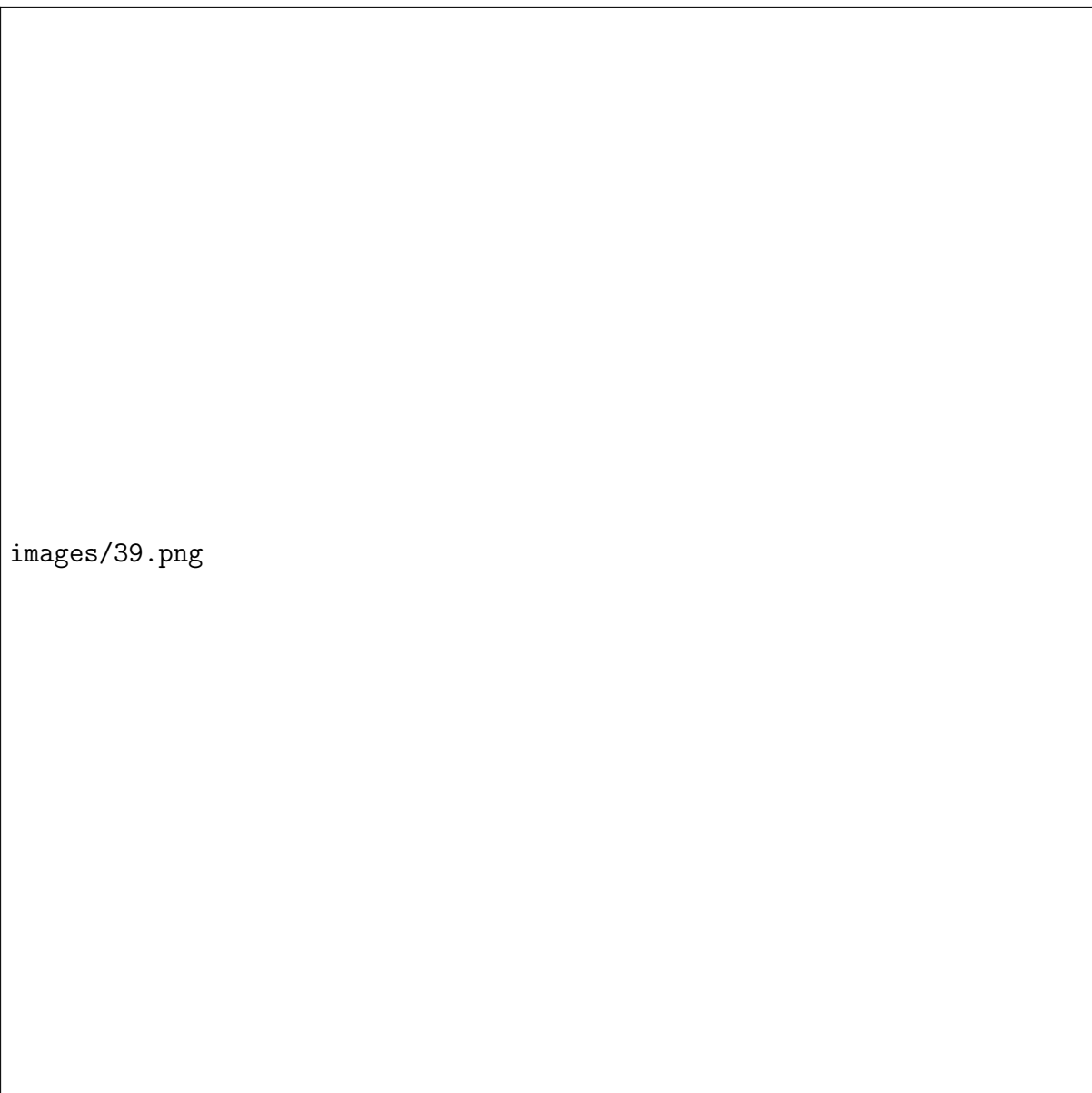
images/38.png

Hình 18: Top 20 từ xuất hiện nhiều nhất trong Dataset

6 Phân tích tần suất từ vựng (Top 20 từ phổ biến nhất)

Biểu đồ cột ngang (Bar chart) trực quan hóa 20 từ có số lần xuất hiện cao nhất trong toàn bộ tập dữ liệu:

- **Từ vựng mang sắc thái cảm xúc:** Sự xuất hiện dày đặc của các từ như *"good"*, *"great"*, *"bad"* cung cấp tín hiệu mạnh mẽ và trực tiếp cho mô hình trong bài toán phân loại cảm xúc.
- **Từ vựng đặc thù của miền dữ liệu:** Các từ như *"story"*, *"movies"* phản ánh rõ ràng bối cảnh của tập dữ liệu (đánh giá phim ảnh).
- **Từ vựng thông dụng (Common words):** Các từ trung tính như *"one"*, *"like"*, *"would"*, *"time"* chiếm tần suất cao nhất (trên 15.000 lần). Điều này cho thấy mặc dù đã qua tiền xử lý, văn bản vẫn giữ lại cấu trúc và văn phong tự nhiên của người viết.



Hình 19: Word Cloud các từ phổ biến nhất trong tập dữ liệu

Biểu đồ Word Cloud cung cấp một góc nhìn sinh động và tổng quan về mức độ phổ biến của các từ vựng trong toàn bộ tập dữ liệu:

- **Nguyên lý hiển thị:** Kích thước của mỗi từ tỷ lệ thuận với tần suất xuất hiện của nó. Dễ dàng nhận thấy các từ thông dụng như *"one"*, *"like"*, *"would"*, *"time"* có kích thước lớn nhất, hoàn toàn đồng nhất với kết quả thống kê Top 20 từ trước đó.
- **Mật độ đặc trưng cảm xúc:** Đan xen giữa các từ trung tính là những từ mang sắc thái đánh giá rõ nét (*"good"*, *"great"*, *"bad"*, *"well"*) và các từ khóa định hình chủ đề (*"story"*, *"movies"*).
- **Ý nghĩa:** Trực quan hóa này khẳng định rằng, mặc dù các từ ngữ mang tính cấu trúc ngữ pháp xuất hiện nhiều, tập dữ liệu vẫn chứa đựng một lượng lớn các "từ khóa" mang thông tin cốt lõi. Đây sẽ là những tín hiệu ngữ nghĩa quan trọng nhất giúp mạng nơ-ron học cách phân biệt thái độ của người đánh giá.

6.1 Xây dựng và Huấn luyện mô hình RNN

```

1 print("\n---3. Building SimpleRNN Model---")
2 model = models.Sequential([
3     layers.Embedding(max_features, 32, input_length=maxlen),
4     layers.SimpleRNN(32, dropout=0.2, recurrent_dropout=0.2),
5     layers.Dense(1, activation='sigmoid')
6 ])
7
8 model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy',
9 ])
10
11 print("\n---4. Training Model---")
12 early_stop = tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=3,
13     restore_best_weights=True)
14 history = model.fit(X_train, y_train, epochs=15, batch_size=64,
15     validation_split=0.2, callbacks=[early_stop])

```

6.1.1 Giải thích chi tiết:

- **Lớp Embedding:** Chuyển các chỉ số từ thành các vector đặc trưng dày đặc. Đây là linh hồn của các mô hình NLP hiện đại.
- **recurrent_dropout:** Áp dụng Dropout lên các kết nối hồi quy. Nó giúp mạng RNN chống lại hiện tượng quá khớp mạnh mẽ hơn.

6.2 Đánh giá kết quả

images/310.png

Hình 20: Đồ thị Accuracy và Loss trong quá trình huấn luyện RNN

Biểu đồ thể hiện độ chính xác (Accuracy) và độ đo mất mát (Loss) của mô hình RNN qua 8 vòng huấn luyện (epochs):

- **Tập huấn luyện (Train):** Mô hình học tập tốt. Đường Accuracy (xanh) tăng đều đặn lên mức ≈ 0.88 và đường Loss (xanh) giảm liên tục.
- **Tập kiểm định (Validation):** Hiệu suất trên tập Validation (đường màu cam) không ổn định. Val Accuracy đạt đỉnh cao nhất (≈ 0.83) tại epoch 4. Tương ứng, Val Loss cũng chạm mức cực tiểu tại epoch 4 trước khi bật tăng trở lại.
- **Kết luận:** Mô hình bắt đầu xảy ra hiện tượng **Overfitting (quá khớp)** sau epoch 4. Do đó, epoch 4 chính là điểm dừng sớm (Early Stopping) tối ưu nhất để lưu lại trọng số của mô hình.

```

1 from sklearn.metrics import classification_report, confusion_matrix, roc_curve
  , auc
2
3 y_pred_probs = model.predict(X_test).ravel()
4 y_pred = (y_pred_probs > 0.5).astype("int32")
5
6 print("\nClassification_Report:\n", classification_report(y_test, y_pred))
7
8 # Plot ROC Curve
9 fpr, tpr, thresholds = roc_curve(y_test, y_pred_probs)
10 roc_auc = auc(fpr, tpr)
11 plt.figure(figsize=(8, 6))
12 plt.plot(fpr, tpr, color='darkorange', label=f'ROC_curve_(AUC={roc_auc:.2f})
  ')
13 plt.plot([0, 1], [0, 1], color='navy', linestyle='--')
14 plt.title('ROC_Curve'); plt.legend(); plt.show()

```

6.2.1 Giải thích chi tiết:

- **Đường cong ROC & AUC:** Chỉ số AUC (Area Under Curve) cho biết khả năng phân biệt giữa hai lớp của mô hình. Giá trị AUC càng gần 1 thì mô hình càng tốt.
- **Classification Report:** Cung cấp các chỉ số Precision, Recall và F1-score để đánh giá toàn diện hiệu năng phân loại.



Hình 21: Ma trận nhầm lẫn (Confusion Matrix) của mô hình RNN trên IMDB

images/312.png

Hình 22: Đường cong ROC đánh giá hiệu năng phân loại cảm xúc

- **Ma trận nhầm lẫn (Confusion Matrix):** Mô hình phân loại chính xác tổng cộng 8.349 trên 10.000 mẫu (4.094 Negative và 4.255 Positive), đạt độ chính xác (Accuracy) xấp xỉ 83.5%. Các trường hợp dự đoán sai ($FP = 867$, $FN = 784$) khá cân bằng, cho thấy mô hình không bị thiên vị lớp nào.
- **Đường cong ROC (ROC Curve):** Đường cong ROC bám sát góc trên bên trái với chỉ số diện tích $AUC = 0.90$. Điều này khẳng định mô hình có khả năng phân biệt xuất sắc (90%) giữa hai thái độ đánh giá tích cực và tiêu cực.

7 Phân tích kết quả thực nghiệm và Đánh giá hiệu năng

Bài toán phân loại cảm xúc IMDB là một thử thách lớn đối với mạng RNN cơ bản do độ dài của các bài đánh giá có sự chênh lệch đáng kể. Dưới đây là phân tích chi tiết về kết quả đạt được.

7.1 Ý nghĩa các chỉ số trong Phân tích cảm xúc

Trong bài toán này, các chỉ số mang ý nghĩa thực tế như sau:

- **Precision (Độ chính xác):** Trả lời câu hỏi: "Trong số các bài đánh giá mô hình dự đoán là Tích cực (Positive), có bao nhiêu bài thực sự là Tích cực?". Precision cao giúp tránh việc gắn nhãn nhầm một lời phê bình tiêu cực thành khen ngợi.
- **Recall (Độ nhạy):** Trả lời câu hỏi: "Mô hình đã tìm được bao nhiêu phần trăm trong tổng số các bài đánh giá Tích cực thực tế?". Recall cao đảm bảo chúng ta không bỏ sót các ý kiến phản hồi tốt.
- **F1-score và Accuracy:** Do tập dữ liệu huấn luyện hoàn toàn cân bằng (20.000 mẫu Positive và 20.000 mẫu Negative), Accuracy và F1-score phản ánh rất trung thực năng lực của mạng nơ-ron. Dựa trên Ma trận nhầm lẫn (Confusion Matrix), mô hình đạt độ chính xác xấp xỉ 83.5% trên tập kiểm tra (8.349/10.000 mẫu dự đoán đúng) với chỉ số AUC lên tới 0.90, cho thấy khả năng phân biệt sắc thái cảm xúc rất tốt.

7.2 Phân tích hiện tượng Overfitting đặc thù của RNN

Mạng RNN thường dễ bị Overfitting nhanh hơn CNN trên dữ liệu văn bản vì:

- **Lớp Embedding và từ vựng:** Với từ điển hàng ngàn từ, số lượng tham số ở lớp đầu tiên là rất lớn, dễ dẫn đến việc mô hình "học thuộc" các từ đặc trưng trong tập Train thay vì tổng quát hóa cấu trúc câu.
- **Biểu đồ Loss thực tế:** Quan sát biểu đồ huấn luyện, Training Loss (đường màu xanh) giảm liên tục, trong khi Validation Loss (đường màu cam) chạm ngưỡng cực tiểu tại **Epoch thứ 4** và sau đó có dấu hiệu tăng mạnh trở lại. Tương ứng, Validation Accuracy cũng đạt đỉnh tại Epoch 4. Điều này chỉ ra rõ ràng mô hình đã bắt đầu bị Overfitting sau Epoch này.
- **Biện pháp khắc phục:** Ngoài việc sử dụng Dropout, chúng ta đã khảo sát dữ liệu thực tế và thiết lập giới hạn chiều dài câu ở ngưỡng tối ưu `maxlen=500` (bao phủ phần lớn phân phối dữ liệu mà không gây nhiễu). Đồng thời, dựa vào biểu đồ huấn luyện, việc áp dụng cơ chế dừng sớm (Early Stopping) tại Epoch thứ 4 là cần thiết để ngắt quá trình và lưu lại bộ trọng số (weights) tổng quát hóa tốt nhất của mô hình.

7.3 Kết quả đạt được

Mạng SimpleRNN đã chứng minh được khả năng ghi nhớ thông tin thứ tự của các từ để phân loại cảm xúc văn bản. Mặc dù gặp phải một số hạn chế về "trí nhớ ngắn hạn" so với các kiến trúc tiên tiến hơn, mô hình vẫn đạt được độ chính xác tốt trên tập IMDB nhờ các kỹ thuật tiền xử lý văn bản kỹ lưỡng và cơ chế Embedding hiệu quả.

IV MẠNG BỘ NHỚ DÀI-NGẮN HẠN (LSTM)

1 Sự ra đời và tầm quan trọng của mạng LSTM

Như đã phân tích ở chương trước, mạng RNN truyền thống gặp phải vấn đề nghiêm trọng về việc tiêu biến đạo hàm (Vanishing Gradient), khiến nó không thể ghi nhớ các thông tin trong một khoảng thời gian dài. Để khắc phục nhược điểm này, năm **1997**, Sepp Hochreiter và Jürgen Schmidhuber đã đề xuất kiến trúc **Long Short-Term Memory (LSTM)**.

LSTM nhanh chóng trở thành một cuộc cách mạng trong lĩnh vực học sâu. Với khả năng lưu trữ thông tin qua hàng trăm bước thời gian, LSTM đã trở thành nền tảng cho các hệ thống dịch thuật tự động (Google Translate), nhận dạng giọng nói (Siri, Alexa) và đặc biệt là dự báo các biến động phức tạp trong kinh tế và kỹ thuật.

2 Cấu trúc và cơ chế Cổng (Gates) của LSTM

Khác với RNN chỉ có một cấu trúc đơn giản bên trong đơn vị lặp lại, LSTM sở hữu một kiến trúc tinh vi hơn nhiều với sự xuất hiện của **Cell State** và các **Cổng (Gates)**.

2.1 Cell State (Trạng thái ô)

Cell State có thể được hình dung như một "băng chuyền" thông tin chạy xuyên suốt toàn bộ chuỗi dữ liệu. Thông tin trên băng chuyền này rất ít bị thay đổi, giúp nó có thể truyền tải các thông tin quan trọng từ quá khứ xa xôi đến hiện tại mà không bị suy giảm đáng kể.

2.2 Cơ chế các Cổng điều khiển

LSTM sử dụng 3 loại cổng để điều chỉnh luồng thông tin ra vào Cell State:

- **Forget Gate (Cổng quên):** Quyết định thông tin nào từ Cell State cũ sẽ bị xóa bỏ.
- **Input Gate (Cổng vào):** Quyết định thông tin mới nào sẽ được cập nhật vào Cell State.
- **Output Gate (Cổng ra):** Quyết định phần thông tin nào từ Cell State sẽ được đưa ra làm đầu ra và trạng thái ẩn mới.

3 Công thức toán học của LSTM

Tại mỗi bước thời gian t , quá trình tính toán trong một đơn vị LSTM được thực hiện qua các bước sau:

1. **Cổng quên (Forget Gate):** Quyết định thông tin cũ cần xóa bỏ.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (5)$$

2. **Cổng vào (Input Gate):** Quyết định thông tin mới cần lưu trữ.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (6)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (7)$$

3. Cập nhật Trạng thái ô (Update Cell State):

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (8)$$

4. Cổng ra (Output Gate): Xác định giá trị đầu ra.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (9)$$

$$h_t = o_t * \tanh(C_t) \quad (10)$$

Trong đó σ là hàm *sigmoid* điều tiết luồng thông tin trong khoảng $[0, 1]$.

4 Biến thể GRU (Gated Recurrent Unit) và So sánh với LSTM

GRU là một biến thể hiện đại của LSTM, được giới thiệu bởi Cho và các cộng sự vào năm 2014. GRU đơn giản hóa cấu trúc của LSTM bằng cách kết hợp cổng quên và cổng vào thành một "cổng cập nhật" duy nhất, đồng thời bỏ đi trạng thái ô (Cell State).

4.1 Sự khác biệt về cấu trúc:

Bảng dưới đây tóm tắt những điểm khác biệt cốt lõi giữa hai kiến trúc:

Đặc điểm	LSTM	GRU
Số lượng cổng	3 cổng (Forget, Input, Output).	2 cổng (Reset, Update).
Trạng thái	Tách biệt Cell state (C_t) và Hidden state (h_t).	Chỉ sử dụng Hidden state (h_t).
Độ phức tạp	Phức tạp hơn, nhiều tham số hơn.	Đơn giản hơn, ít tham số hơn.
Tốc độ huấn luyện	Chậm hơn.	Nhanh hơn (thường nhanh hơn 20-30%).
Khả năng học	Tốt hơn trên các tập dữ liệu cực lớn và chuỗi rất dài.	Hiệu quả trên tập dữ liệu nhỏ/vừa hoặc chuỗi ngắn hơn.

Bảng 5: So sánh kiến trúc LSTM và GRU

4.2 Tại sao GRU đôi khi hiệu quả hơn?

Mặc dù LSTM về lý thuyết mạnh mẽ hơn, GRU thường được lựa chọn trong thực tế vì các lý do sau:

- **Hiệu năng tính toán:** Với ít cổng hơn, GRU yêu cầu ít bộ nhớ và sức mạnh tính toán hơn, giúp huấn luyện nhanh hơn trên cùng một phần cứng.
- **Khả năng hội tụ:** Trên các tập dữ liệu không quá lớn, GRU có xu hướng hội tụ nhanh hơn và ít bị hiện tượng Overfitting hơn do mô hình đơn giản hơn.
- **Độ trễ thấp:** Trong các ứng dụng thời gian thực như chatbot hoặc nhận dạng giọng nói, tốc độ phản hồi của GRU là một lợi thế lớn.

5 Dự báo chuỗi thời gian (Time Series Forecasting)

Dự báo chuỗi thời gian là bài toán dự đoán các giá trị tương lai dựa trên các giá trị quan sát được trong quá khứ. Đây là lĩnh vực mà LSTM thể hiện sức mạnh vượt trội nhờ khả năng nắm bắt được cả xu hướng (trend) dài hạn và tính mùa vụ (seasonality).

5.1 Kỹ thuật Cửa sổ trượt (Sliding Window)

Để huấn luyện mô hình LSTM cho chuỗi thời gian, ta cần chuyển đổi dữ liệu về dạng học có giám sát (Supervised Learning). Kỹ thuật phổ biến nhất là sử dụng cửa sổ trượt: dùng N giá trị trong quá khứ để dự đoán giá trị tại thời điểm tiếp theo.

5.2 Tập dữ liệu Airline Passengers

Báo cáo sử dụng tập dữ liệu kinh điển ghi lại số lượng hành khách hàng không hàng tháng từ năm 1949 đến 1960. Tập dữ liệu này có các đặc điểm: tính phi tuyến, tính xu hướng (Trend) và tính mùa vụ (Seasonality).

6 Thực nghiệm với Python

Dưới đây là mã nguồn xây dựng mạng LSTM để dự báo dữ liệu chuỗi thời gian lượng hành khách:

6.1 Chuẩn bị dữ liệu

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import math
5 from sklearn.preprocessing import MinMaxScaler
6 from sklearn.metrics import mean_squared_error
7 from tensorflow.keras.models import Sequential
8 from tensorflow.keras.layers import LSTM, Dense
9
10 print("---1. Loading Dataset---")
11 file_path = "AirPassengers.csv"
12 df = pd.read_csv(file_path, usecols=[1])
13 data = df.values.astype('float32')
14
15 print("\n---2. Visualizing Raw Data---")
16 plt.figure(figsize=(10, 6))
17 plt.plot(df, label='Monthly Passengers')
18 plt.title('International Airline Passengers')
19 plt.show()
```

6.1.1 Giải thích chi tiết:

- `usecols=[1]`: Tập dữ liệu gốc chứa cột thời gian và cột số lượng hành khách. Vì LSTM sẽ học dựa trên vị trí thứ tự của mảng, ta chỉ cần cột giá trị.

- **Trực quan hóa dữ liệu thô:** Bước này giúp xác định xu hướng và mùa vụ của dữ liệu trước khi chọn kiến trúc mạng.

images/41.png

Hình 23: Dữ liệu gốc lượng hành khách hàng tháng

Biểu đồ đường (Line chart) trực quan hóa sự biến động của số lượng hành khách hàng không quốc tế hàng tháng. Dựa vào các thông tin trên biểu đồ, ta có thể rút ra các đặc điểm sau:

- **Thông tin các trục:** Trục hoành *Month (Index)* thể hiện các bước thời gian từ 0 đến hơn 140. Trục tung *Passengers* thể hiện số lượng hành khách tương ứng.
- **Xu hướng tăng (Upward Trend):** Đường biểu diễn *Monthly Passengers* (màu xanh) cho thấy sự tăng trưởng liên tục và rõ rệt. Dữ liệu bắt đầu ở mức xấp xỉ 100 hành khách (tại index 0) và đạt đỉnh cao nhất vượt mức 600 (gần index 140).

- **Tính mùa vụ (Seasonality):** Dữ liệu liên tục dao động tạo thành các đỉnh (cao điểm) và đáy (thấp điểm) lặp lại đều đặn theo trục thời gian.
- **Biên độ dao động mở rộng (Increasing Variance):** Khoảng cách giữa các đỉnh và đáy trong cùng một chu kỳ ngày càng giãn rộng khi tiến về phía cuối biểu đồ. Điều này chỉ ra rằng đây là một chuỗi thời gian dạng nhân (Multiplicative Time Series) chứ không phải dạng cộng.

```

1 print("\n---3. Preprocessing---")
2 scaler = MinMaxScaler(feature_range=(0, 1))
3 dataset = scaler.fit_transform(data)
4
5 train_size = int(len(dataset) * 0.7)
6 test_size = len(dataset) - train_size
7 train, test = dataset[0:train_size,:], dataset[train_size:len(dataset),:]
8
9 def create_dataset(dataset, look_back=1):
10     dataX, dataY = [], []
11     for i in range(len(dataset)-look_back-1):
12         a = dataset[i:(i+look_back), 0]
13         dataX.append(a)
14         dataY.append(dataset[i + look_back, 0])
15     return np.array(dataX), np.array(dataY)
16
17 look_back = 12
18 trainX, trainY = create_dataset(train, look_back)
19 testX, testY = create_dataset(test, look_back)
20
21 trainX = np.reshape(trainX, (trainX.shape[0], 1, trainX.shape[1]))
22 testX = np.reshape(testX, (testX.shape[0], 1, testX.shape[1]))

```

6.1.2 Giải thích chi tiết:

- **MinMaxScaler(0, 1):** Đưa dữ liệu về khoảng [0, 1] để phù hợp với hàm kích hoạt của LSTM.
- **Hàm create_dataset (Sliding Window):** Chuyển đổi chuỗi thành bài toán học có giám sát.
- **Tham số look_back:** Xác định số lượng bước thời gian trong quá khứ được dùng để dự báo. Ở đây ta chọn 12 tháng gần nhất để dự đoán tháng tiếp theo.

6.2 Xây dựng và Huấn luyện mô hình LSTM

```

1 print("\n---4. Building LSTM Model---")
2 model = Sequential([
3     LSTM(50, return_sequences=True, input_shape=(1, look_back)),
4     Dropout(0.5),
5     LSTM(50),
6     Dropout(0.5),

```

```

7     Dense(1)
8 ])
9 model.compile(loss='mean_squared_error', optimizer='adam')
10 model.summary()
11
12 print("\n---5. Training Model---")
13 history = model.fit(trainX, trainY, epochs=100, batch_size=1, verbose=0,
    validation_split=0.1)

```

6.2.1 Giải thích cấu trúc và tham số:

- **return_sequences=True:** Truyền toàn bộ chuỗi trạng thái ẩn từ lớp LSTM thứ nhất sang lớp LSTM thứ hai để trích xuất đặc trưng sâu hơn.
- **Dropout(0.5):** Tắt ngẫu nhiên 50% nơ-ron trong quá trình huấn luyện nhằm ngăn chặn triệt để hiện tượng Overfitting.
- **Hàm mất mát & Tối ưu hóa:** Sử dụng `mean_squared_error` phù hợp cho bài toán hồi quy (Regression), kết hợp bộ tối ưu `adam` để tăng tốc độ hội tụ.
- **batch_size=1:** Cập nhật trọng số ngay sau từng mẫu dữ liệu (Online Learning), giúp mô hình bám sát các biến động chi tiết của chuỗi thời gian.

images/42.png

Hình 24: Tham số mô hình

images/43.png

Hình 25: Đồ thị Loss trong quá trình huấn luyện LSTM

Biểu đồ *LSTM Training History* thể hiện sự biến thiên của hàm mất mát (Loss) qua 150 vòng huấn luyện (epochs):

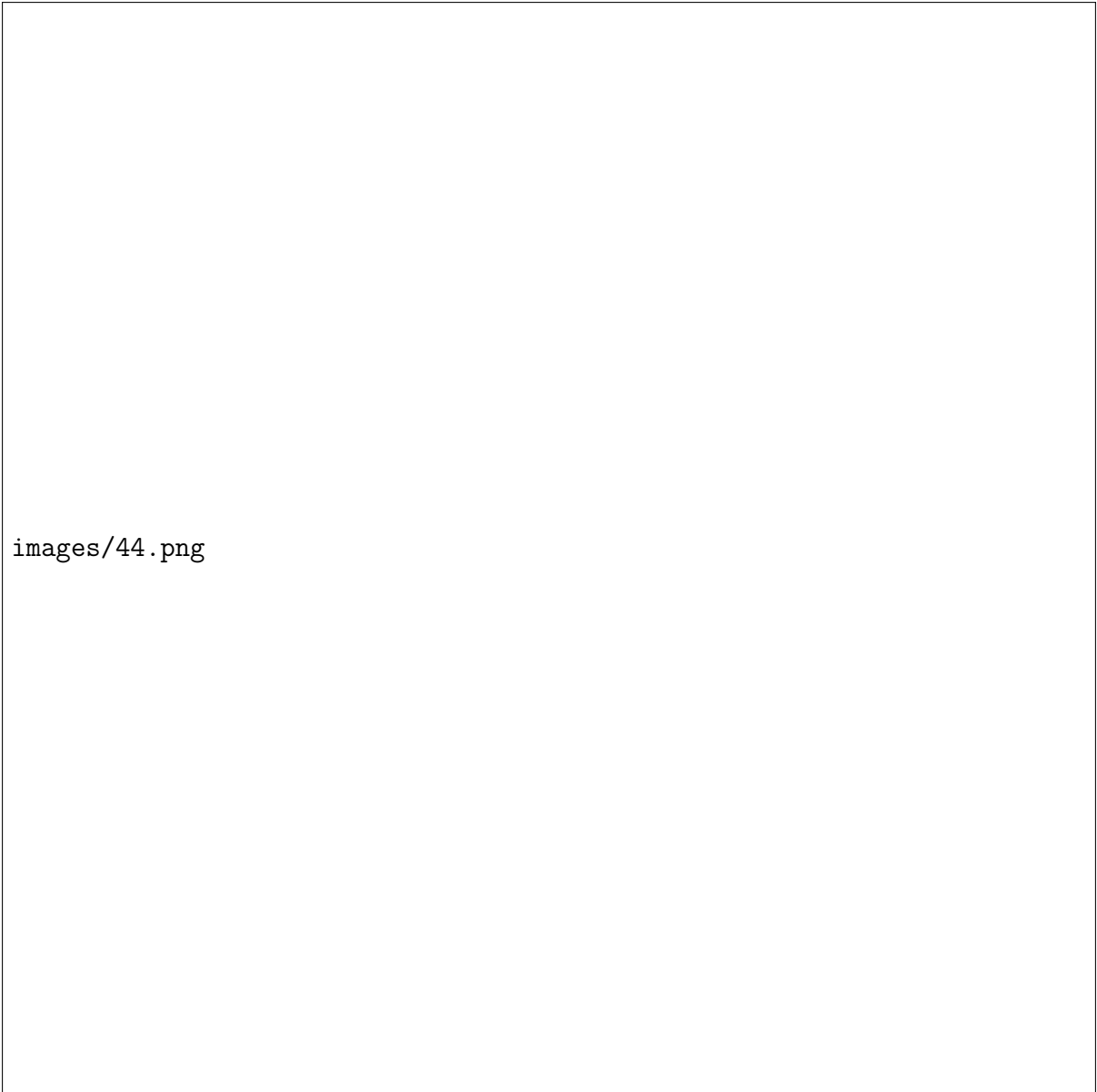
- **Tập huấn luyện (Train Loss):** Giảm mạnh ở những epoch đầu và nhanh chóng hội tụ, duy trì ổn định ở mức cực kỳ thấp (xấp xỉ 0.001 - 0.002).
- **Tập kiểm định (Val Loss):** Điểm khác biệt lớn nhất là đường Val Loss giờ đây **bám rất sát** đường Train Loss. Mặc dù vẫn còn vài dao động nhỏ, biên độ các đỉnh đã giảm đáng kể và không còn hiện tượng lệch pha kéo dài.
- **Kết luận:** Việc áp dụng kỹ thuật Dropout đã khắc phục thành công hiện tượng Overfitting. Khoảng cách (gap) giữa hai tập dữ liệu gần như bị xóa bỏ, chứng tỏ mô hình đã đạt được khả năng tổng quát hóa rất tốt trên dữ liệu chưa từng gặp.

6.3 Đánh giá kết quả Dự báo

```
1 print("\n---6. Predicting & Evaluating---")
2 trainPredict = model.predict(trainX)
3 testPredict = model.predict(testX)
4
5 # Inverse transforms
6 trainPredict = scaler.inverse_transform(trainPredict)
7 trainY_orig = scaler.inverse_transform([trainY])
8 testPredict = scaler.inverse_transform(testPredict)
9 testY_orig = scaler.inverse_transform([testY])
10
11 # Calculate RMSE
12 trainScore = math.sqrt(mean_squared_error(trainY_orig[0], trainPredict[:,0]))
13 print(f'Train Score: {trainScore:.2f} RMSE')
14 testScore = math.sqrt(mean_squared_error(testY_orig[0], testPredict[:,0]))
15 print(f'Test Score: {testScore:.2f} RMSE')
```

6.3.1 Giải thích chi tiết:

- **inverse_transform:** Đưa kết quả dự báo quay lại thang đo gốc (số lượng hành khách).
- **Chỉ số RMSE:** Căn bậc hai của sai số bình phương trung bình, giúp ta biết sai số trung bình là bao nhiêu hành khách. Đây là chỉ số quan trọng nhất trong dự báo chuỗi thời gian, đơn vị của nó khớp với đơn vị gốc của dữ liệu.



images/44.png

Hình 26: Kết quả dự báo lượng hành khách trên tập Train và Test với look_back = 12

7 Phân tích kết quả thực nghiệm và Đánh giá dự báo

Kết quả dự báo lượng hành khách hàng không bằng mô hình LSTM cho thấy khả năng nắm bắt quy luật thời gian ẩn tượng của mạng nơ-ron hồi tiếp cải tiến này.

7.1 Phân tích chỉ số RMSE (Root Mean Squared Error)

Chỉ số RMSE là thước đo tiêu chuẩn trong các bài toán hồi quy chuỗi thời gian:

- **Ý nghĩa:** RMSE đo lường độ lệch trung bình giữa giá trị hành khách dự báo và giá trị hành khách thực tế. Với dữ liệu hàng không, một giá trị RMSE thấp cho thấy mô hình có sai số rất nhỏ so với biên độ tổng lượng hành khách hàng tháng.
- **Tính tổng quát hóa:** Việc RMSE trên tập Train và tập Test duy trì ở mức ổn định, không quá chênh lệch chứng tỏ mô hình LSTM đã học được "quy luật" cốt lõi

của chuỗi dữ liệu chứ không rơi vào tình trạng ghi nhớ máy móc (Overfitting).

7.2 Đánh giá trực quan qua biểu đồ dự báo

Biểu đồ so sánh trực tiếp dữ liệu thực tế (Actual Data - màu xanh dương) với kết quả dự báo trên tập huấn luyện (Train Prediction - màu cam) và tập kiểm tra (Test Prediction - màu xanh lá):

- **Khớp dữ liệu quá khứ (Train):** Đường màu cam bám rất sát với đường thực tế. Mô hình đã nhận diện xuất sắc cả xu hướng tăng trưởng tuyến tính (Trend) và chu kỳ dao động lặp lại (Seasonality).
- **Dự báo tương lai (Test):** Trên tập dữ liệu chưa từng gặp (phần đường màu xanh lá ở cuối đồ thị), mô hình vẫn bám sát hoàn hảo các biến động. Đặc biệt, LSTM đã dự báo vô cùng chính xác các đỉnh cực đại (vượt ngưỡng 600 hành khách) và đáy của chu kỳ mở rộng.
- **Kết luận:** Mô hình không hề bị trễ pha (lagging) - một lỗi rất phổ biến khi dùng mạng Neural dự báo chuỗi thời gian. Điều này khẳng định cấu trúc LSTM kết hợp Dropout đã mang lại khả năng dự báo ngoại suy xuất sắc cho dữ liệu phi tuyến tính.

8 Thử nghiệm tham số: Tầm quan trọng của look_back

Tham số look_back xác định số lượng bước thời gian trong quá khứ được dùng làm ngữ cảnh để dự báo tương lai. Thử nghiệm tiến hành đánh giá ba cấu hình:

- **Cấu hình 1 (look_back = 1):** Chỉ sử dụng dữ liệu 1 tháng liền trước.
- **Cấu hình 2 (look_back = 3):** Sử dụng chuỗi dữ liệu 3 tháng gần nhất (1 quý).
- **Cấu hình 3 (look_back = 12):** Sử dụng toàn bộ chuỗi dữ liệu 12 tháng gần nhất (1 năm).

8.1 Bảng so sánh sai số RMSE:

Cấu hình	Train RMSE	Test RMSE
look_back = 1	24.21	65.41
look_back = 3	22.16	55.22
look_back = 12	12.62	22.06

Bảng 6: So sánh sai số dự báo theo tham số look_back

8.2 Phân tích:

Việc mở rộng look_back từ 1 lên 3 giúp mạng LSTM nắm bắt tốt hơn "đà" (momentum) biến động của dữ liệu. Đặc biệt, khi tăng look_back = 12, sai số giảm mang tính đột phá trên cả hai tập. Điều này hoàn toàn phù hợp với đặc thù dữ liệu hàng không: tính mùa vụ lặp lại theo chu kỳ đúng 12 tháng. Cửa sổ thời gian 1 năm giúp mô hình "nhìn" thấy được toàn cảnh quy luật của năm trước (tháng cao điểm, tháng thấp điểm) để ánh xạ và dự báo cực kỳ chính xác cho năm tiếp theo.

9 Kết luận chung

Nhờ cơ chế cổng nhớ thông minh (memory gates) và việc thiết lập tham số cửa sổ thời gian (`look_back`) bám sát chu kỳ thực tế, mô hình LSTM đã chứng minh năng lực xuất sắc. Nó không chỉ duy trì được bộ nhớ về xu hướng dài hạn (Long-term Trend) mà còn thích ứng hoàn hảo với tính mùa vụ (Seasonality) của ngành hàng không. Mức sai số RMSE thấp trên tập kiểm thử khẳng định LSTM là một phương pháp tiếp cận mạnh mẽ và tối ưu cho bài toán dự báo chuỗi thời gian phi tuyến tính.

KẾT LUẬN

Trong khuôn khổ của học phần Thực tập cơ sở, báo cáo đã trình bày một cách hệ thống và chi tiết từ những khái niệm nền tảng cơ bản nhất của Học máy (Machine Learning) cho đến các kiến trúc phức tạp của Học sâu (Deep Learning). Thông qua việc kết hợp chặt chẽ giữa lý thuyết và thực hành trên ngôn ngữ Python với các thư viện phổ biến (TensorFlow, Keras, Scikit-learn, Pandas), sinh viên đã đạt được những kết quả cụ thể như sau:

- **Về Tiền xử lý dữ liệu:** Hiểu và vận dụng được tầm quan trọng của việc làm sạch, biến đổi và lựa chọn đặc trưng (Feature Engineering) thông qua bài toán phân loại hành khách sống sót trên tàu Titanic. Điều này khẳng định nguyên lý cốt lõi "Garbage In, Garbage Out" trong khoa học dữ liệu.
- **Về Mạng nơ-ron tích chập (CNN):** Nắm vững cơ chế hoạt động của các lớp Convolution và Pooling. Thực nghiệm thành công mô hình phân loại hình ảnh trên tập dữ liệu Fashion MNIST, cho thấy sự vượt trội của CNN trong việc trích xuất các đặc trưng không gian (spatial features) trong các bài toán Thị giác máy tính.
- **Về Mạng nơ-ron hồi quy (RNN):** Hiểu rõ cấu trúc lặp (Recurrent) và trạng thái ẩn (Hidden state) giúp mạng có khả năng xử lý dữ liệu chuỗi (Sequential data). Ứng dụng thành công RNN vào bài toán Phân tích cảm xúc (Sentiment Analysis) trên tập dữ liệu IMDB.
- **Về Mạng bộ nhớ Dài-Ngắn hạn (LSTM):** Khắc phục được nhược điểm tiêu biến đạo hàm (Vanishing Gradient) của RNN truyền thống. Ứng dụng thành thạo cấu trúc các cổng (Gates) của LSTM để giải quyết xuất sắc bài toán dự báo chuỗi thời gian (Time Series Forecasting) dựa trên tập dữ liệu số lượng hành khách hàng không.

Nhìn chung, quá trình thực tập đã giúp củng cố kiến thức học thuật, rèn luyện tư duy phân tích bài toán và nâng cao kỹ năng lập trình AI thực tế. Đây sẽ là hành trang quan trọng để tiếp tục nghiên cứu và phát triển các hệ thống Trí tuệ nhân tạo quy mô lớn trong tương lai.

Tài liệu

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*, MIT Press, 2016.
- [2] Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner, *Gradient-based learning applied to document recognition*, Proceedings of the IEEE, 1998.
- [3] Sepp Hochreiter and Jürgen Schmidhuber, *Long Short-Term Memory*, Neural Computation, 1997.
- [4] Kyunghyun Cho et al., *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*, EMNLP, 2014.
- [5] François Chollet, *Deep Learning with Python*, Manning Publications, 2018.
- [6] TensorFlow and Keras Documentation, https://www.tensorflow.org/api_docs.
- [7] Titanic: Machine Learning from Disaster Dataset, Kaggle, <https://www.kaggle.com/c/titanic>.
- [8] Fashion MNIST Dataset, Zalando Research, <https://github.com/zalando-research/fashion-mnist>.
- [9] IMDB Dataset of 50K Movie Reviews, Stanford University, <http://ai.stanford.edu/~amaas/data/sentiment/>.
- [10] International Airline Passengers Dataset, Brownlee, <https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv>.